ECI 39:
Exploiting and securing communication on the Internet
Day #2

PKI, TLS/SSL, HTTP and DNS

Vladimír Veselý, veselyv@fit.vut.cz

28th July 2026

L7 and L4 bootstrapping

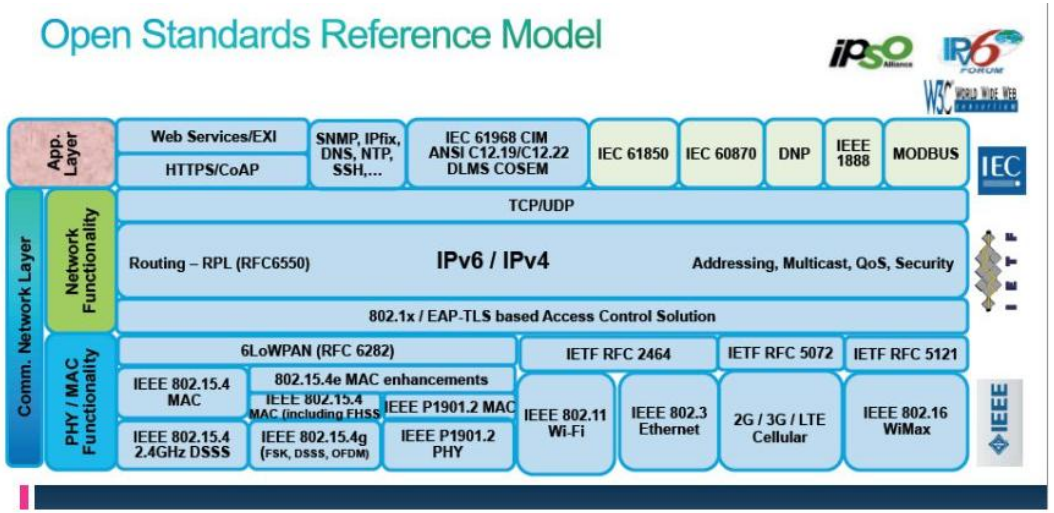
APPLICATIONS AND PORTS

Application Protocol

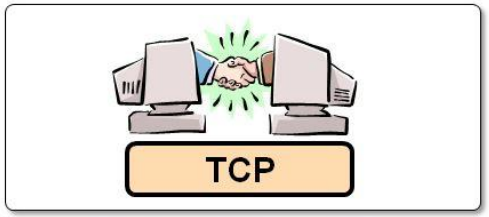
- Messages that are being exchanged
  - E.g., request and response
  - Rules when and how are message exchanged
- Syntax = fields in the header's PDU
- Semantics = meaning of the message and its effect on protocol machine

- Text vs. Binary

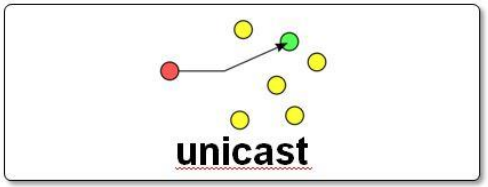
- Public-domain protocols:
  - Defined in RFC
  - Interoperability
  - HTTP, SMTP, FTP, POP3, RIP, BGP, TCP, UDP
- Proprietary protocols:
  - KaZaA, ICQ, Skype, WhatsApp
  - IGRP, EIGRP



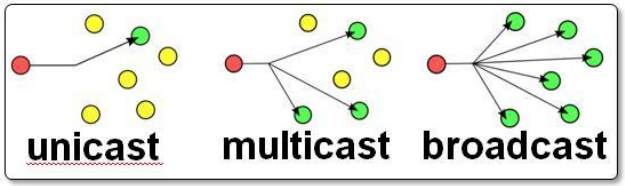
Transport Protocols (1)



- Slower but reliable transfers
- Typical applications:
  - Email
  - Web browsing



- Fast but non-guaranteed transfers ("best effort")
- Typical applications:
  - VoIP
  - Music streaming



TCP	UDP
Keeps track of lost packets. Makes sure that lost packets are re-sent	Doesn't keep track of lost packets
Adds sequence numbers to packets and reorders any packets that arrive in the wrong order	Doesn't care about packet arrival order
Slower, because of all added additional functionality	Faster, because it lacks any extra features
Requires more computer resources, because the OS needs to keep track of ongoing communication sessions and manage them on a much deeper level	Requires less computer resources
Examples of programs and services that use TCP: - HTTP - HTTPS - FTP - Many computer games	Examples of programs and services that use UDP: - DNS - IP telephony - DHCP - Many computer games

Internet Applications

Application	Protocol	Port Number
File Transfer Protocol FTP Client	TCP	20
File Transfer Protocol FTP Server	TCP	21
Secure Shell SSH	TCP	22
Telnet	TCP	23
Simple Mail Transport Protocol SMTP	TCP	25
Domain Name System DNS	UDP / TCP	53
Dynamic Host Configuration Protocol DHCP	UDP	67,68
Trivial File Transfer Protocol TFTP	UDP	69
Hypertext Transfer Protocol HTTP	TCP	80
Post Office Protocol 3 POP3	TCP	110
Simple Network Management Protocol SNMP	UDP	161
Hypertext Tranfer Protocol Secure HTTPS	TCP	443

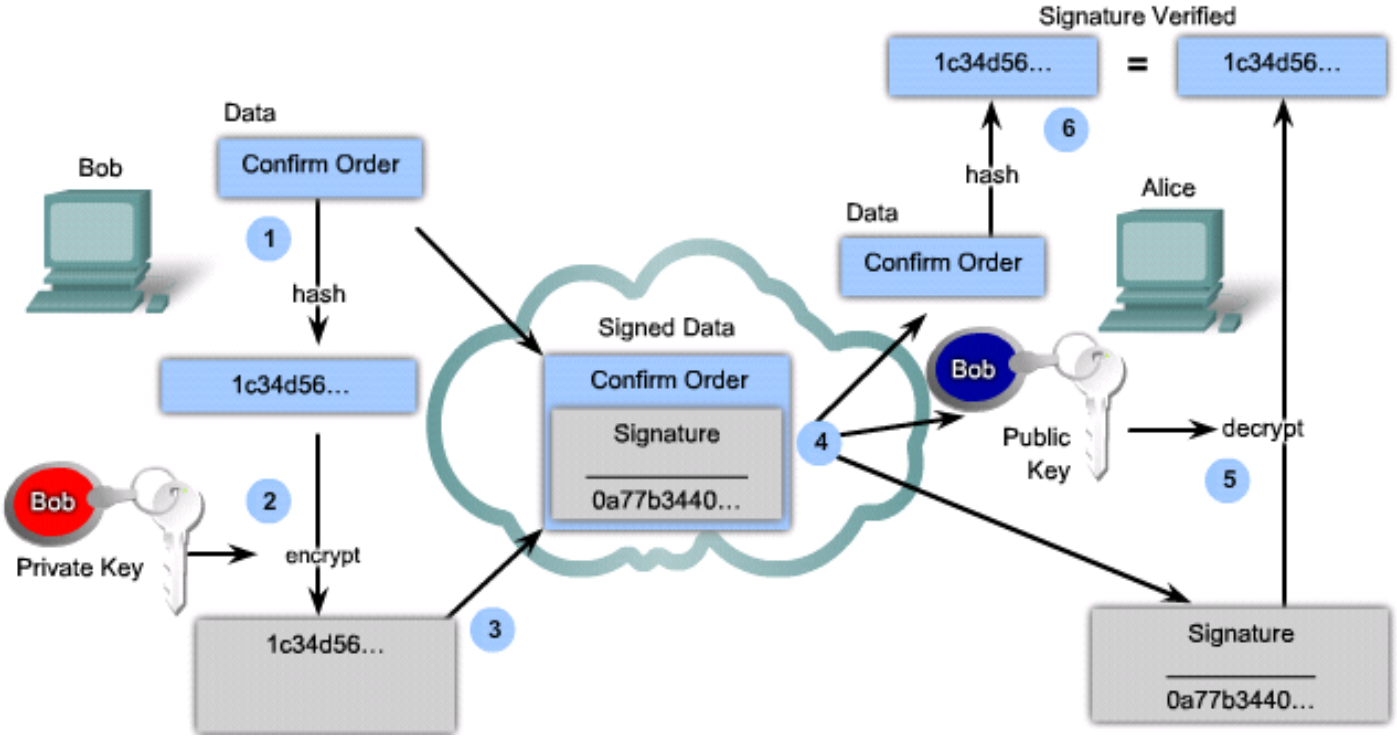
Cryptology in daily praxis

PUBLIC KEY INFRASTRUCTURE

Digital Signatures

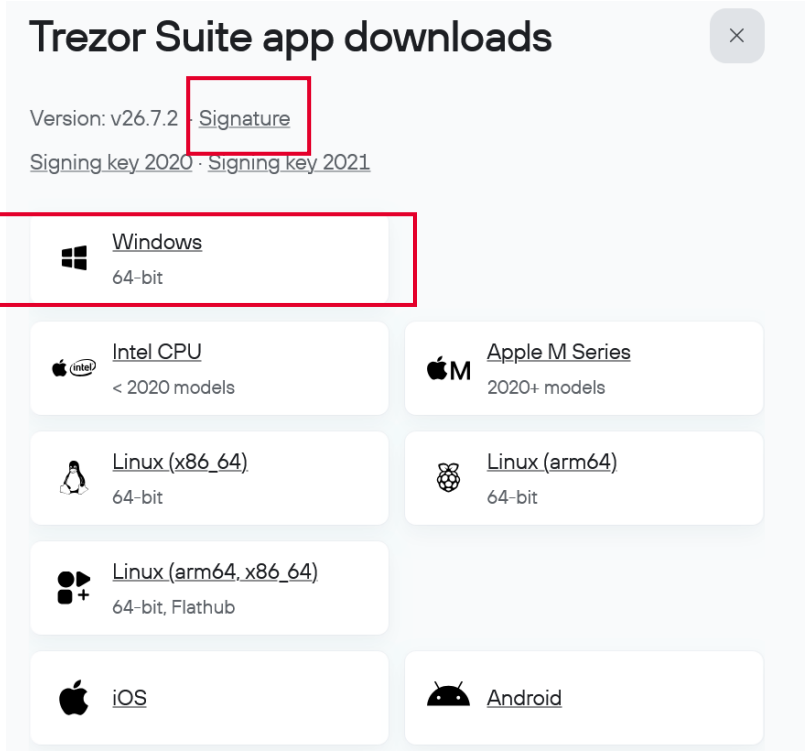
- Digital signatures are often used in the following situations:
  - To provide a unique proof of data source, which can only be generated by a single party, such as contract signing in e-commerce environments.
  - To authenticate a user by using the private key of that user and the signature it generates.
  - To prove the authenticity and integrity of PKI certificates.
  - To provide nonrepudiation using a secure timestamp and a trusted time source.
    - Each party has a unique, secret signature key, which is not shared with any other party, making nonrepudiation possible.
- Authenticity of digitally signed data:
  - Digital signatures authenticate a source, proving that a certain party has seen and signed the data in question.
- Integrity of digitally signed data:
  - Digital signatures guarantee that the data has not changed from the time it was signed.
- Nonrepudiation of the transaction:
  - The recipient can take the data to a third party, and the third party accepts the digital signature as a proof that this data exchange did take place.
  - The signing party cannot repudiate that it has signed the data.

Digital Signatures



1) Bob creates a hash of the document.
2) Bob encrypts the hash with the private key.
3) The encrypted hash, known as the signature, is appended to the document.
4) Alice accepts the document with the digital signature and obtains Bob's public key
5) Alice decrypts the signature using Bob's public key to unveil the assumed hash value
6) Alice calculates the hash of the received document, without its signature, and compares this hash to the decrypted signature hash and if the hashes match = document is authentic.

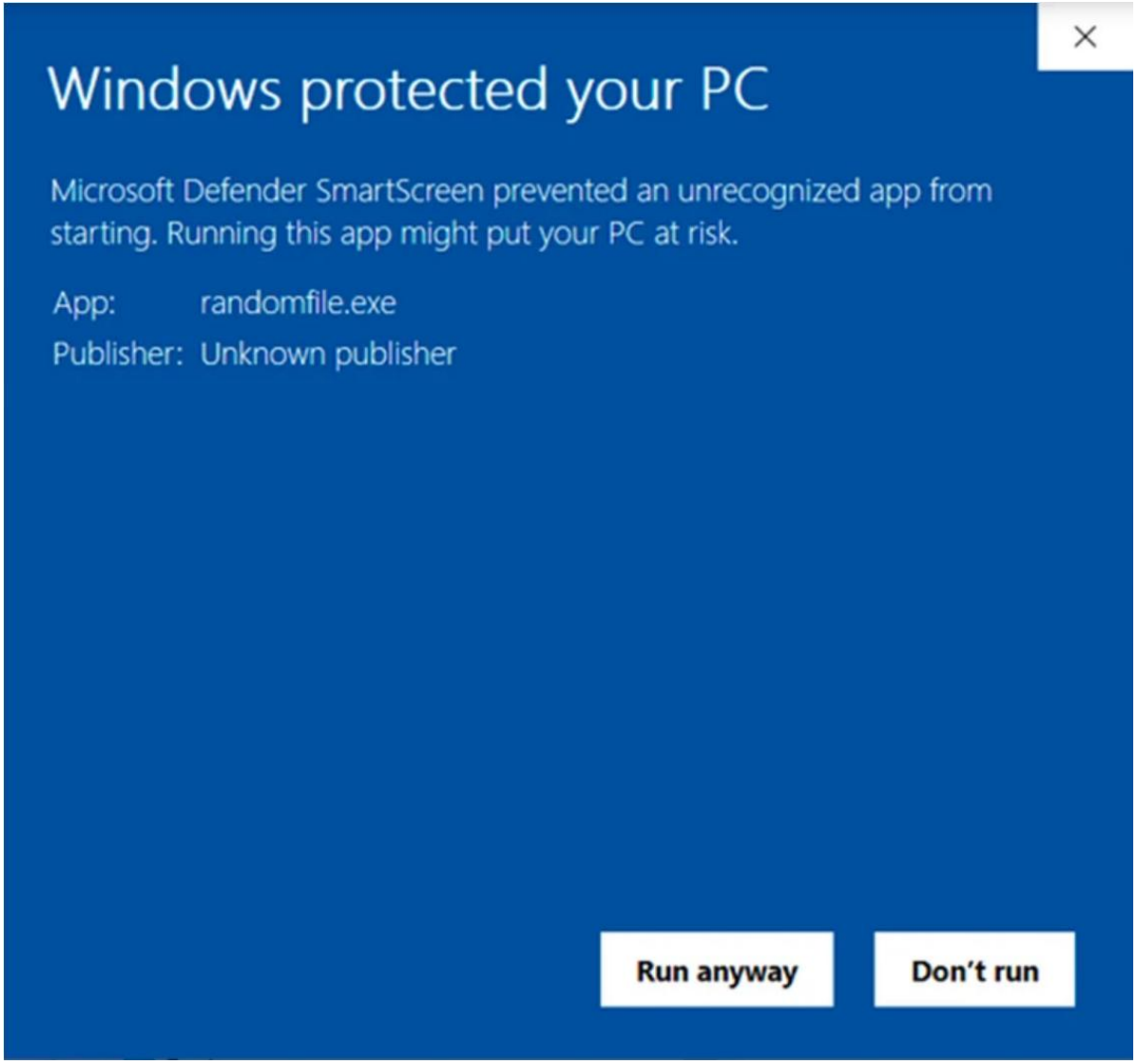
Example of Signed Software



- `gpg --verify "Trezor-Suite-26.7.2-win-x64.exe.asc" "Trezor-Suite-26.7.2-win-x64.exe"`

```
gpg: Signature made 07/22/26 10:16:47 Argentina Standard Time
gpg:                using RSA key EB483B26B078A4AA1B6F425EE21B6950A2ECB65C
gpg: Good signature from "SatoshiLabs 2021 Signing Key" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
    EB483B26B078A4AA1B6F425EE21B6950A2ECB65C
```

Do NOT Underestimate Similar Warnings



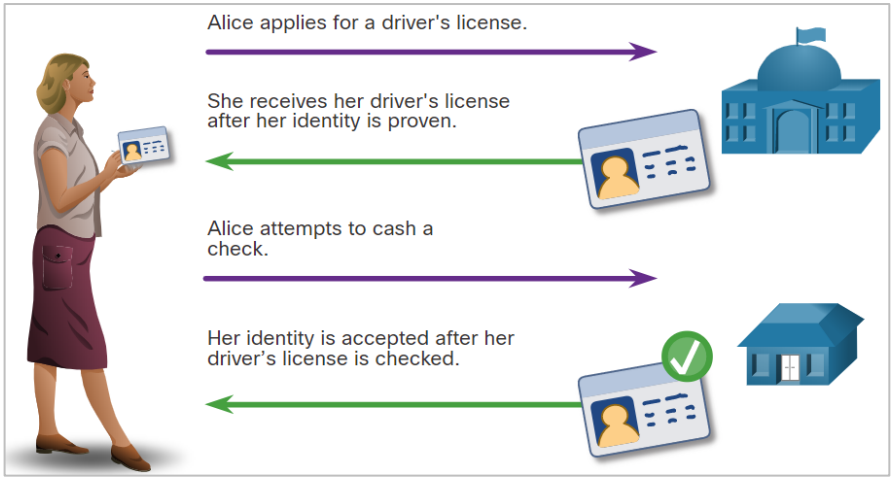
Algorithms for Signing

Algorithm	Description, Author, and Key Sizes
DSA (Digital Signature Algorithm)	Developed by the NSA and adopted by NIST as part of the Digital Signature Standard (DSS). Uses modular exponentiation and discrete logarithms. Key sizes: 1024 to 3072 bits. Commonly used in government and secure communication protocols.
RSA (Rivest–Shamir–Adleman)	Developed by Ron Rivest, Adi Shamir, and Leonard Adleman. Based on the difficulty of factoring large integers. Supports both encryption and digital signatures. Key sizes: 1024 to 4096 bits. Widely used in SSL/TLS and digital certificates.
ECDSA (Elliptic Curve Digital Signature Algorithm)	Developed by Neal Koblitz and Victor S. Miller. An elliptic curve variant of DSA, providing the same security with smaller key sizes. Key sizes: 160 to 521 bits. Commonly used in modern cryptography, including Bitcoin and TLS.
Ed25519	Developed by Daniel J. Bernstein. A high-performance elliptic curve digital signature algorithm. Known for speed and security. Uses a 256-bit key (Ed25519 curve). Popular in OpenSSH and modern cryptographic protocols.
Schnorr Signature	Developed by Claus Schnorr. A digital signature scheme known for simplicity and efficiency. Recently adopted in Bitcoin (Taproot upgrade). Key sizes: 256 bits (commonly with elliptic curves).
GOST R 34.10	Developed by the Russian Federal Agency for Technical Regulation and Metrology. A digital signature algorithm based on elliptic curve cryptography. Key sizes: 256 bits. Used in Russian government standards for secure communication.
Falcon	Developed by Pierre-Alain Fouque, Thomas Prest, and others. A lattice-based post-quantum digital signature algorithm. Designed for security against quantum computers. Key sizes: 512 to 1024 bits. A finalist in the NIST post-quantum cryptography competition.
Rainbow	Developed by Jintai Ding and Dieter Schmidt. A multivariate polynomial-based digital signature algorithm. Post-quantum secure. Key sizes: Varies depending on security level, typically large due to multivariate nature.

Public Key Management

- When establishing an asymmetric connection between two hosts, the hosts will exchange their public key information.
  - Trusted third parties on the Internet validate the authenticity of these public keys using digital certificates. The third-party issues credentials that are difficult to forge.
  - From that point forward, all individuals who trust the third party simply accept the credentials that the third-party issues.
- **The Public Key Infrastructure (PKI) consists of specifications, systems, and tools that are used to create, manage, distribute, use, store, and revoke digital certificates.**
  - The Certificate Authority (CA) creates digital certificates by tying a public key to a confirmed identify, such as a website or individual.

- *Illustrates how a driver's license is analogous to a digital certificate*



PKI Components (1)

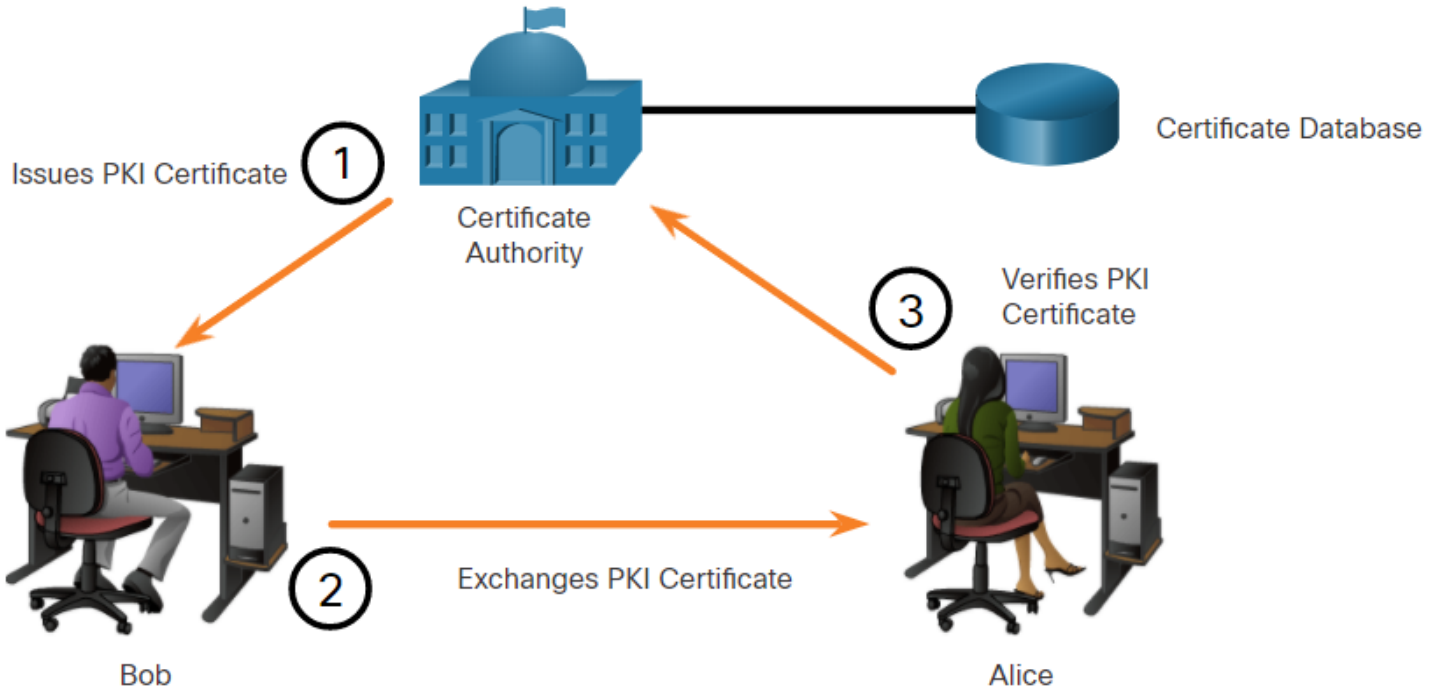
- PKI is needed to support large-scale distribution and identification of public encryption keys.

1. PKI certificates contain an entity's or individual's public key, its purpose, the certificate authority (CA) that validated and issued the certificate, the date range during which the certificate is valid, and the algorithm used to create the signature.
2. The certificate store resides on a local computer and stores issued certificates and private keys.
3. The PKI Certificate of Authority (CA) is a trusted third party that issues PKI certificates to entities and individuals after verifying their identity. It signs these certificates using its private key.
4. The certificate database stores all certificates approved by the CA.

PKI Components (2)

Component	Detailed Description
Root Certificate Authority (Root CA)	The highest level of trust in a PKI hierarchy. A Root CA is a trusted authority that issues certificates to subordinate CAs, creating a chain of trust. The Root CA's public key is distributed and stored in web browsers, operating systems, and devices by default. If the Root CA is compromised, the entire trust chain is broken. Examples: DigiCert, GlobalSign, Entrust, Let's Encrypt, and Sectigo.
Certificate Authority (CA)	A trusted entity responsible for issuing, managing, and revoking digital certificates. The CA verifies the identity of applicants and digitally signs their certificates using its private key. CAs can be classified into Root CAs and Intermediate CAs.
Intermediate Certificate Authority (Intermediate CA)	Acts as a bridge between the Root CA and end-entity certificates. It is authorized by a Root CA and can issue certificates to organizations or individuals. Intermediate CAs help compartmentalize risk; if compromised, only its issued certificates are affected.
Registration Authority (RA)	An RA acts as an intermediary between the certificate applicant and the CA. It validates the applicant's identity and verifies the information provided in the Certificate Signing Request (CSR). Once verified, it forwards the request to the CA for issuance. RAs are commonly used by large organizations for internal certificate management.
Digital Certificate	An electronic document binding a public key to the identity of an individual, organization, or device. It includes: Public Key, Certificate Holder's Identity, CA's Digital Signature, Certificate Validity Period, and Serial Number. Digital certificates are used for SSL/TLS, email encryption, document signing, and code signing.
Public and Private Keys	The key pair at the core of asymmetric cryptography: • Public Key: Shared openly, used for encryption or signature verification. • Private Key: Kept secret, used for decryption or signing. Public keys are distributed via digital certificates, and private keys are stored securely (e.g., in Hardware Security Modules (HSMs)).
Certificate Revocation List (CRL)	A list published by the CA containing digital certificates that have been revoked before their expiration date. Reasons for revocation: Private Key Compromise, Certificate Misuse, or Organizational Changes. Users check the CRL to ensure a certificate is valid before secure communication.
Online Certificate Status Protocol (OCSP)	A real-time protocol for checking the status of a digital certificate without downloading the entire CRL. Clients send an OCSP request to the CA's OCSP responder, which returns the certificate status (Good, Revoked, or Unknown). OCSP is faster and more efficient than CRLs, making it suitable for web browsers and real-time applications.

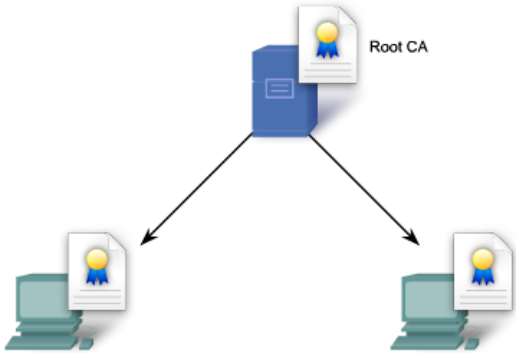
PKI Components (3)



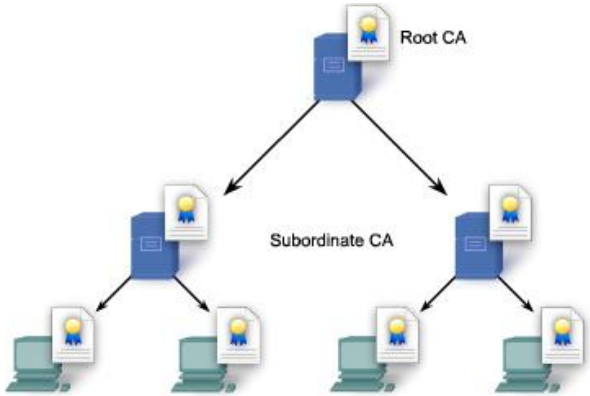
1. **Issues PKI Certificate**. Bob initially requests a certificate from the CA. The CA authenticates Bob and stores Bob's PKI certificate in the certificate database.
2. **Exchanges PKI Certificate**. Bob communicates with Alice using his PKI certificate.
3. **Verifies PKI Certificate**. Alice communicates with the trusted CA using the CA's public key. The CA refers to the certificate database to validate Bob's PKI certificate.

- PKIs can form different **topologies / levels of trust** which are as follows

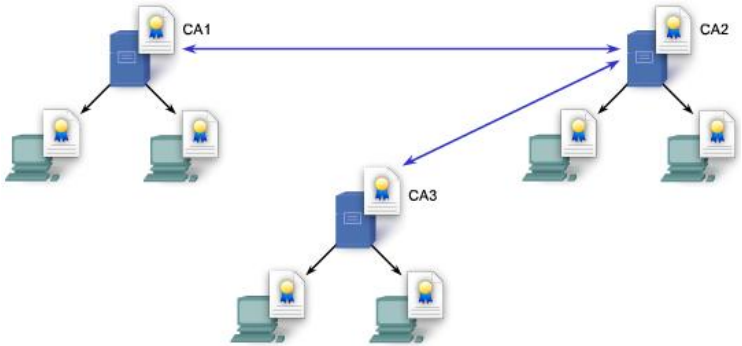
- **Single-Root PKI Topology**: The simplest is the single-root PKI topology. The root CA issues all the certificates to the end users within the same organization. On larger networks, PKI CAs may be linked using two basic architectures:
  - **Hierarchical CA topologies**: The root CA (highest level CA), can issue certificates to end users and to a subordinate CA.
  - **Cross-certified CA topologies**: A peer-to-peer model in which individual CAs establish trust relationships with other CAs by cross-certifying CA certificates.



Single-Root PKI Topology



Hierarchical CA Topologies



Cross-certified CA Topologies

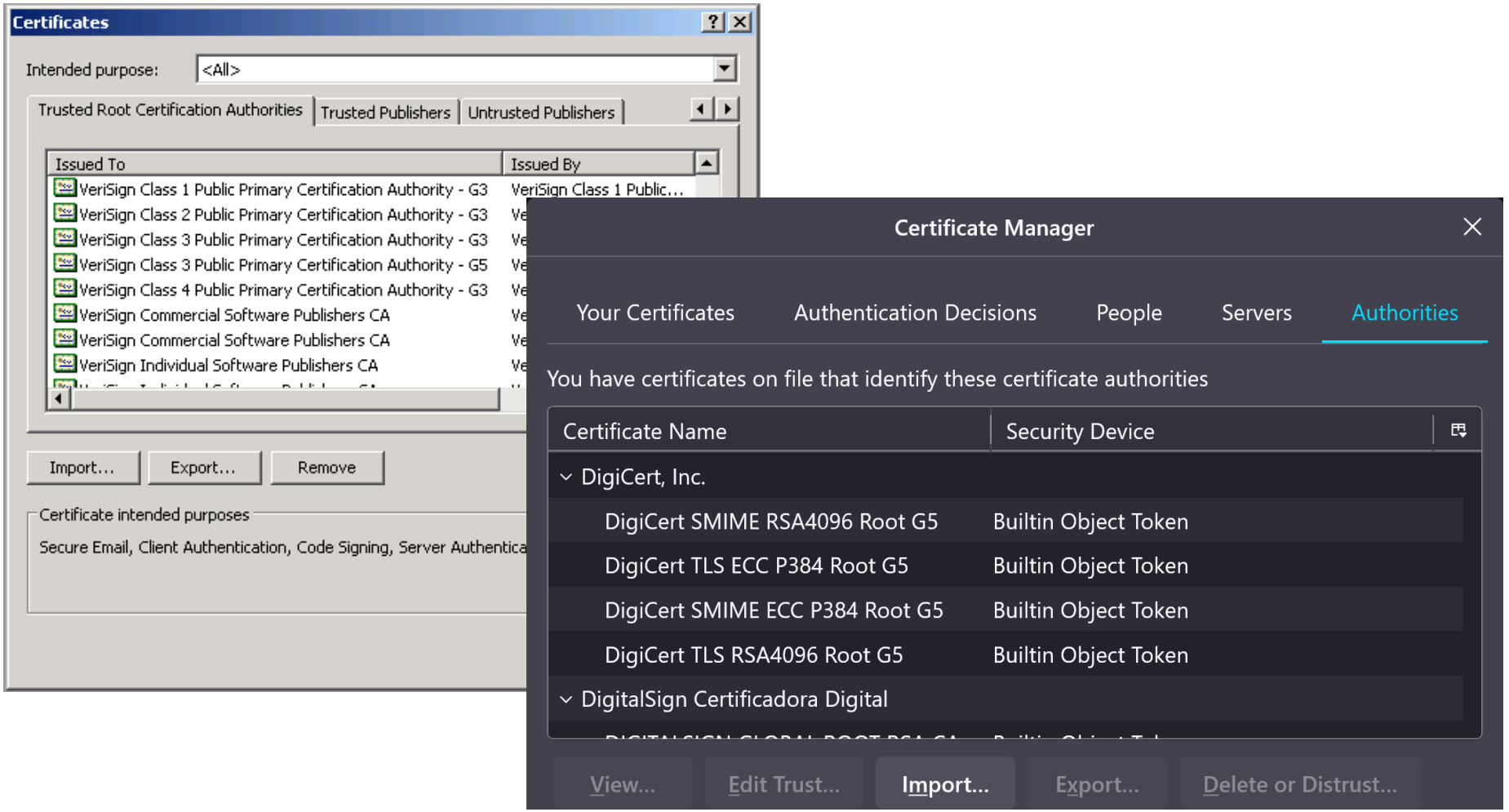
The PKI Authorities System

- Many vendors provide CA servers as a managed service or as an end-user product.
  - Organizations may also implement private PKIs using Microsoft Server or Open SSL.
- CAs issue certificates based on classes which determine how trusted a certificate is.
  - The class number is determined by how rigorous the procedure was that verified the identity of the holder when the certificate was issued.
  - The higher the class number, the more trusted the certificate.

Class	Description
0	Used for testing in situations in which no checks have been performed.
1	Used by individuals who require verification of email.
2	Used by organizations for which proof of identity is required.
3	Used for servers and software signing.
4	Used for online business transactions between companies.
5	Used for private organizations or government security.

Certificate Stores

- Some CA public keys are preloaded, such as those listed in web browsers.



Vendors Certificate

http://www.verisign.com

http://www.entrust.com

http://www.verizonbusiness.com/

http://www.novell.com

The Security Division of EMC
http://www.rsa.com/

http://www.microsoft.com

Pricing

SSL

Domain Validation (DV) SSL Certificate

EASY TO INSTALL SECURITY

Best for verifying your website with Google, and helping boost Google rankings. You get SHA-2 and 2048-bit encryption and a trust indicator in the address bar.

Learn more about SSL Certificates

AS LOW AS

€54.99 /yr

49% savings with a 3-yr term

See Plans

FULLY MANAGED - FOR YOU

SSL

Managed SSL Service

MANAGED FOR YOU SECURITY

Best for getting an SSL fully installed and managed by us that verifies your site and helps boost Google rankings. Includes SHA-2 and 2048-bit encryption and a trust indicator in the address bar.

AS LOW AS

€84.99 /yr

50% savings with a 2-yr term

See Plans

Website Security

EXTENSIVE SECURITY

Best for helping to secure your site with a firewall, malware scanning, SSL certificate, annual site cleanup, and Blocklist monitoring.

AS LOW AS

€5.49 /mo

50% savings with a 2-yr term

See Plans

Basic - OV

Complete business-level validation, reinforcing customer confidence in online presence while securing customer data

☐ Convert this to a wildcard domain

$26 / month / standard domain

12 month auto-renewing subscription $312.00

ADD TO CART

LEARN MORE >

Includes:

- ✓ Unlimited certificate issuance/replacement
- ✓ 24x5 standard support
- ✓ DigiCert CertCentral management portal

Secure Site - OV

Business-level validation with priority validation, management tools and faster issuance

☐ Convert this to a wildcard domain

$44 / month / standard domain

12 month auto-renewing subscription $528.00

ADD TO CART

LEARN MORE >

Includes:

- ✓ Basic - OV benefits, plus
- ✓ Priority order processing
- ✓ DigiCert Smart Seal
- ✓ Domain reputation monitoring
- ✓ Vulnerability assessment scans

RECOMMENDED

Secure Site Pro - OV

Business-level validation, priority validation, and enhanced security for domain and brand protection

☐ Convert this to a wildcard domain

$108 / month / standard domain

12 month auto-renewing subscription $1,296.00

ADD TO CART

LEARN MORE >

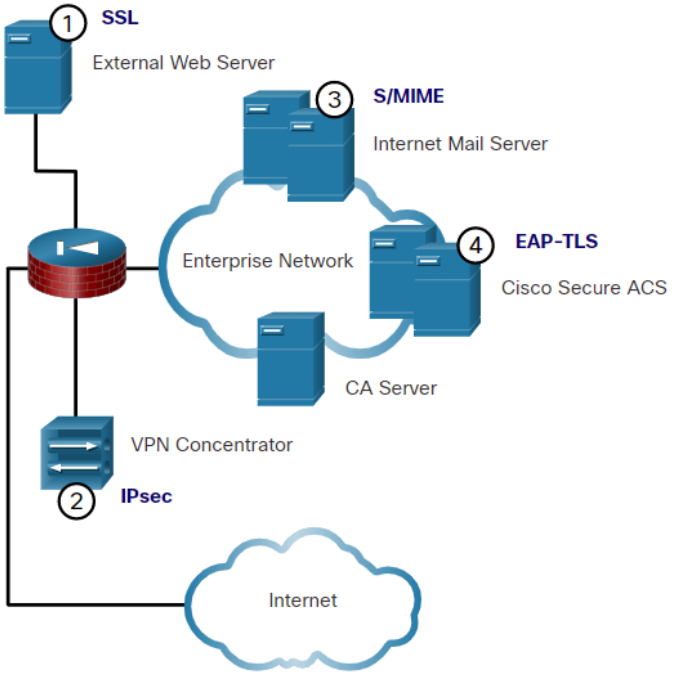
Includes:

- ✓ Basic - OV benefits, plus
- ✓ Priority order processing
- ✓ DigiCert Smart Seal
- ✓ Domain reputation monitoring
- ✓ Vulnerability assessment scans
- ✓ Brand protection (CT log monitoring)

Interoperability of Different PKI Vendors

- Interoperability between a PKI and its supporting services is a concern because many CA vendors have proposed and implemented proprietary solutions.
- To address this interoperability concern, the IETF published the Internet X.509 Public Key Infrastructure Certificate Policy and Certification Practices Framework (RFC 2527).
- **The X.509 version 3 (X.509 v3) standard defines the format of a digital certificate.**

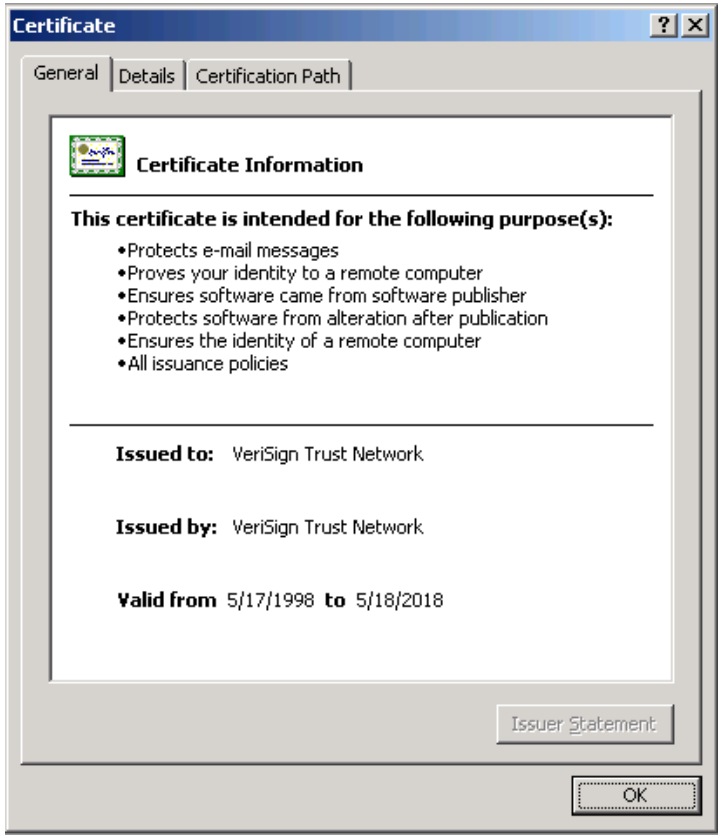
- X.509v3 Applications



1. **SSL** - Secure web servers user X.509.v3 for website authentication in the SSL and TLS protocols, while web browsers use X.509v3 to implement HTTPS client certificates. SSL is the most widely used certificate-based authentication.
2. **IPsec** - IPsec VPNs use X.509 when certificates can be used as a public key distribution mechanism for internet key exchange (IKE) RSA-based authentication.
3. **S/MIME** - User mail agents that support mail protection with the Secure/Multipurpose Internet Mail Extensions (S/MIME) protocol use X.509 certificates.
4. **EAP-TLS** - Cisco switches can use certificates to authenticate end devices that connect to LAN ports using 802.1.x between the adjacent devices. The authentication can be proxied to a central ACS via the Extensible Authentication Protocol with TLS (EAP-TLS).

X.509v3

- Defines basic PKI formats such as the certificate and certificate revocation list (CRL) format to enable basic interoperability



PostSignum Root QCA 4

Subject Name

Country	CZ
	NTRCZ-47114983
Organization	Česká pošta, s.p.
Common Name	PostSignum Root QCA 4

Issuer Name

Country	CZ
	NTRCZ-47114983
Organization	Česká pošta, s.p.
Common Name	PostSignum Root QCA 4

Validity

Not Before	Thu, 26 Jul 2018 09:56:08 GMT
Not After	Mon, 26 Jul 2038 09:56:08 GMT

Public Key Info

Algorithm	RSA
Key Size	4096
Exponent	65537
Modulus	C6:66:8D:82:A0:7E:BE:8B:22:25:78:10:C0:08:8B:9A:19:7F:D5:AD:00:14:0E:64:DE:D...

Miscellaneous

Serial Number	0F:A0
Signature Algorithm	SHA-512 with RSA Encryption
Version	3
Download	PEM (cert) PEM (chain)

Fingerprints

SHA-256	AC:7F:78:62:E6:85:C7:A7:D9:82:6A:58:EA:32:D1:83:D4:89:3F:CC:8F:8F:D6:D9:00:C...
SHA-1	AA:40:D2:57:9B:A8:24:24:CD:27:71:9B:1D:6B:1F:35:71:73:80:99

X509v3 Fields

Field	Description
Version	Specifies the version of the X.509 standard being used (e.g., V1, V2, V3).
Serial Number	A unique identifier assigned to the certificate by the Certificate Authority (CA).
Signature Algorithm	The algorithm used by the CA to sign the certificate (e.g., SHA-256 with RSA).
Issuer	The name of the Certificate Authority (CA) that issued the certificate.
Validity Period	The time frame during which the certificate is valid. Includes: • Not Before: The start date and time when the certificate becomes valid. • Not After: The expiration date and time when the certificate expires.
Subject	The entity that the certificate identifies (e.g., domain name or organization).
Subject Public Key Info	Information about the public key associated with the subject, including: • Public Key Algorithm: The algorithm (e.g., RSA, ECC). • Public Key Value: The actual public key.
Signature Value	The digital signature of the certificate, created by the CA using its private key.
Key Usage (V3 Extension)	Specifies the purposes for which the key may be used (e.g., digital signatures, key encipherment).
Extended Key Usage (V3 Extension)	Additional purposes for the certificate, such as client authentication or code signing.
Basic Constraints (V3 Extension)	Indicates if the certificate is a CA certificate and specifies the path length for certificate chaining.
Subject Alternative Name (SAN)	Lists additional identities for the certificate, such as multiple domain names or IP addresses (common for multi-domain SSL certificates).
Authority Key Identifier (V3 Extension)	Provides a unique identifier for the CA that issued the certificate, linking it to the issuer's public key.
CRL Distribution Points (V3 Extension)	Specifies where to find the CA's Certificate Revocation List (CRL) to check if the certificate has been revoked.
Authority Information Access (AIA)	Provides information about how to access the CA's OCSP responder for real-time certificate status checking.

- The Public-Key Cryptography Standards (PKCS) refers to a group of standards devised and published by RSA Laboratories.
  - PKCS provides basic interoperability of applications that use public-key cryptography.
  - PKCS defines the low-level formats for the secure exchange of arbitrary data, such as an encrypted piece of data or a signed piece of data.

PKCS Standards

PKCS Standard	Title	Description
PKCS #1	**RSA Cryptography Standard**	Defines RSA encryption and signatures, including OAEP for encryption and PSS for signatures.
PKCS #3	**Diffie-Hellman Key Exchange**	Standard for securely establishing shared keys using the Diffie-Hellman protocol.
PKCS #5	**Password-Based Encryption (PBE)**	Describes methods for encrypting private keys using passwords (e.g., PBKDF2 for key derivation).
PKCS #7	**Cryptographic Message Syntax (CMS)**	Standard for digitally signed and encrypted messages, commonly used in S/MIME for secure email.
PKCS #8	**Private Key Information Syntax**	Defines the format for storing private keys (encrypted or unencrypted). Common in .p8 files.
PKCS #9	**Attribute Types for PKCS**	Defines standard attributes (e.g., email, challenge password) used in certificate requests (CSR).
PKCS #10	**Certificate Signing Request (CSR)**	Standard format for requesting a digital certificate from a Certificate Authority (CA).
PKCS #11	**Cryptographic Token Interface**	Standard for accessing cryptographic services on hardware security devices (e.g., HSMs, smart cards).
PKCS #12	**Personal Information Exchange (PFX)**	Format for storing certificates and private keys together (.pfx or .p12 files).
PKCS #13	**Elliptic Curve Cryptography (ECC)**	Describes standards for using ECC for strong security with shorter key lengths.
PKCS #15	**Cryptographic Token Information Format**	Defines the structure for storing cryptographic information on smart cards, including private keys and certificates.

PKCS Filetypes

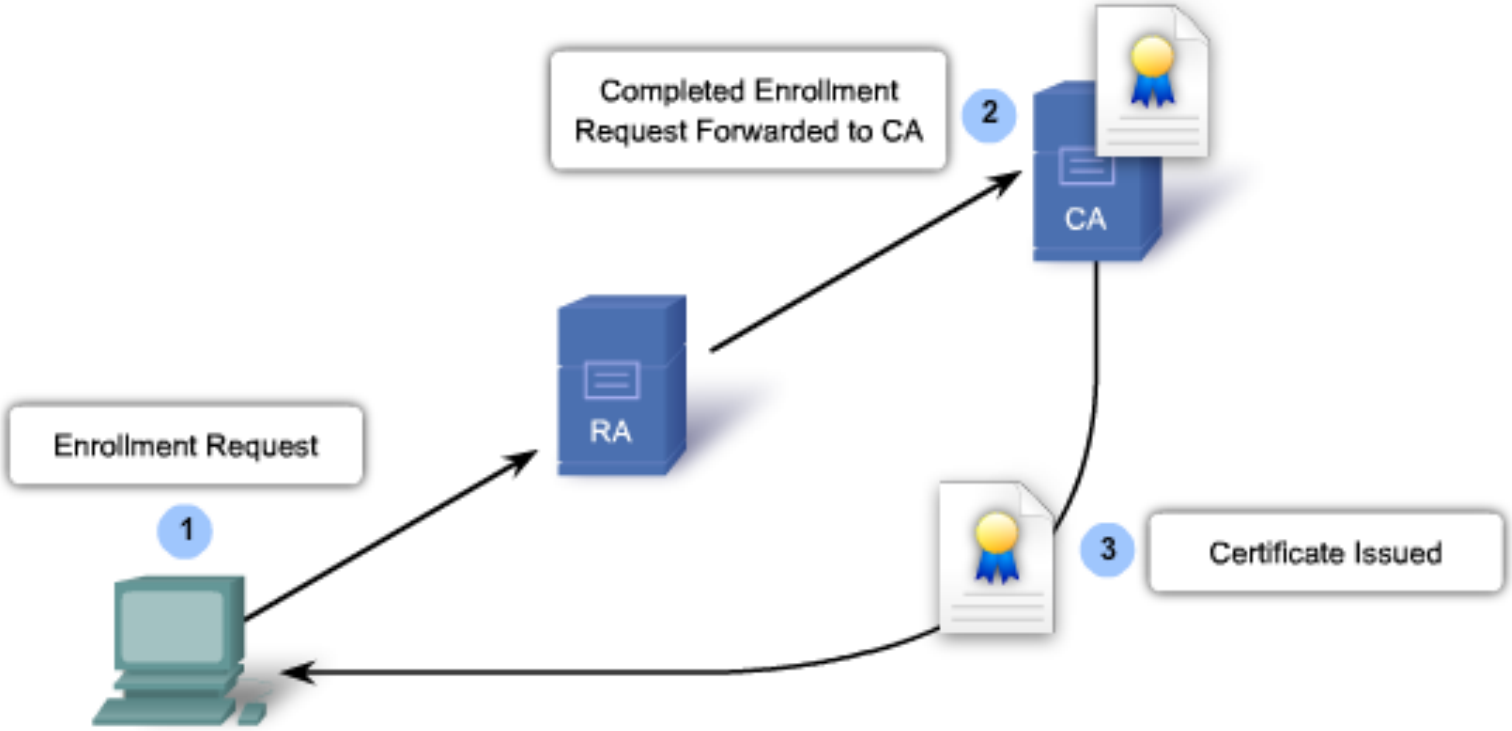
- Covers cryptographic standards beyond just certificates (keys, tokens, encryption).

File Type	Extension	Standard	Description
Certificate Signing Request (CSR)	`.csr`	PKCS #10	Used to request a digital certificate from a CA.
Private Key File	`.p8, .key`	PKCS #8	Stores private keys, optionally encrypted.
Personal Information Exchange	`.pfx, .p12`	PKCS #12	Stores private keys and certificates together.
Cryptographic Message Syntax	`.p7b, .p7c`	PKCS #7	Used for signed or encrypted messages.
Hardware Security Module Interface	N/A	PKCS #11	Interface standard for HSMs and smart cards.

Certificate Enrollment, Authentication and Revocation

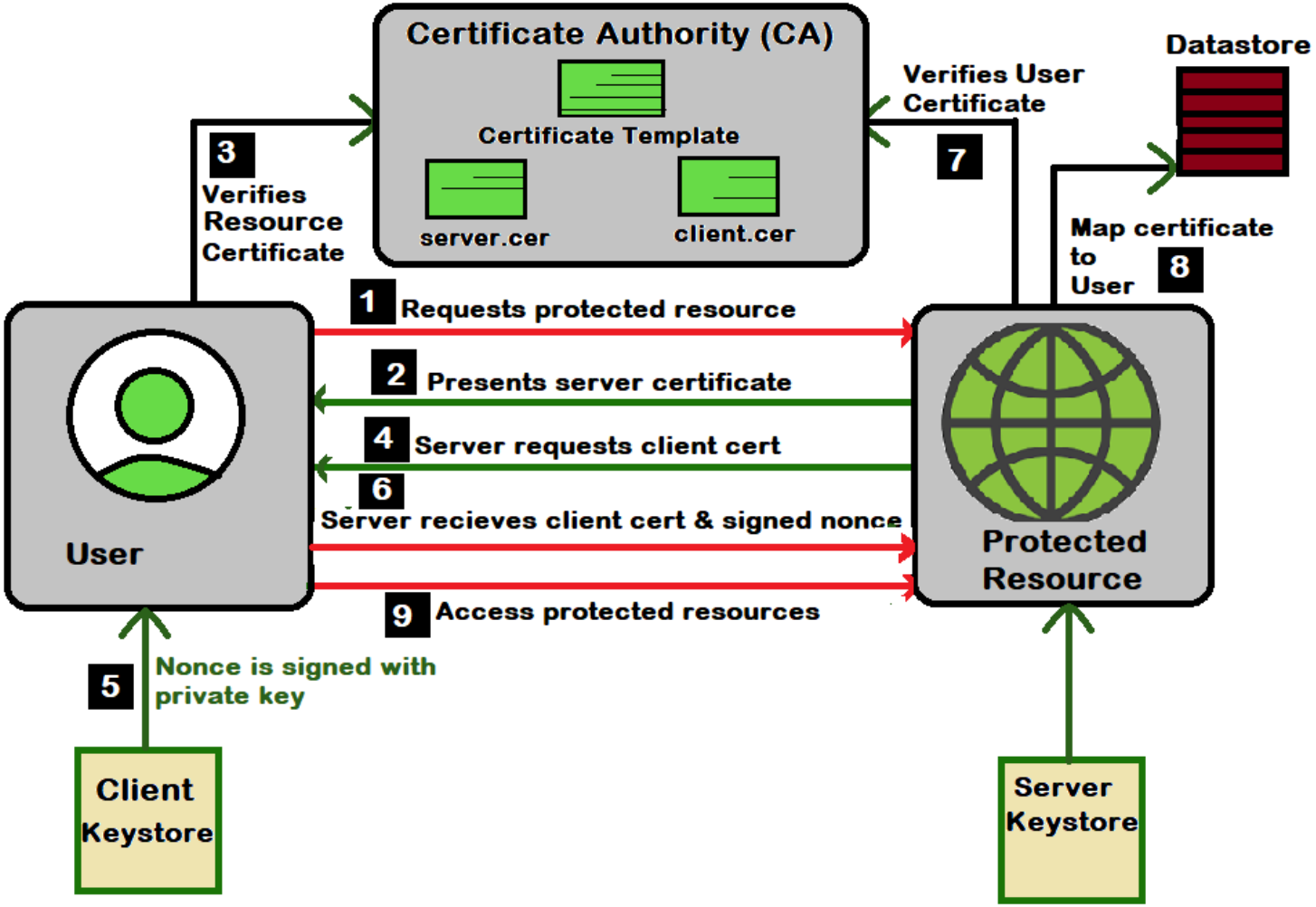
- All systems that leverage the PKI must have the CA's public key.
  - The CA public key verifies all the certificates issued by the CA and is vital for the proper operation of the PKI!
- The certificate enrollment process is used by a host system to enroll with a PKI. To do so, CA certificates are retrieved in-band over a network, and the authentication is done out-of-band (OOB) using the telephone.
  - The system enrolling with the PKI contacts a CA to request and obtain a digital identity certificate for itself and to get the CA's self-signed certificate.
  - The final stage verifies that the CA certificate was authentic and is performed using an out-of-band method such as the POTS to obtain the fingerprint of the valid CA identity certificate.
- A digital certificate can be revoked if key is compromised or if it is no longer needed.
  - Unfortunately, CA can be compromised as well https://en.wikipedia.org/wiki/DigiNotar
- Only a root CA can issue a self-signed certificate that is recognized or verified by other CAs within the PKI.

PKI Enrollment Process



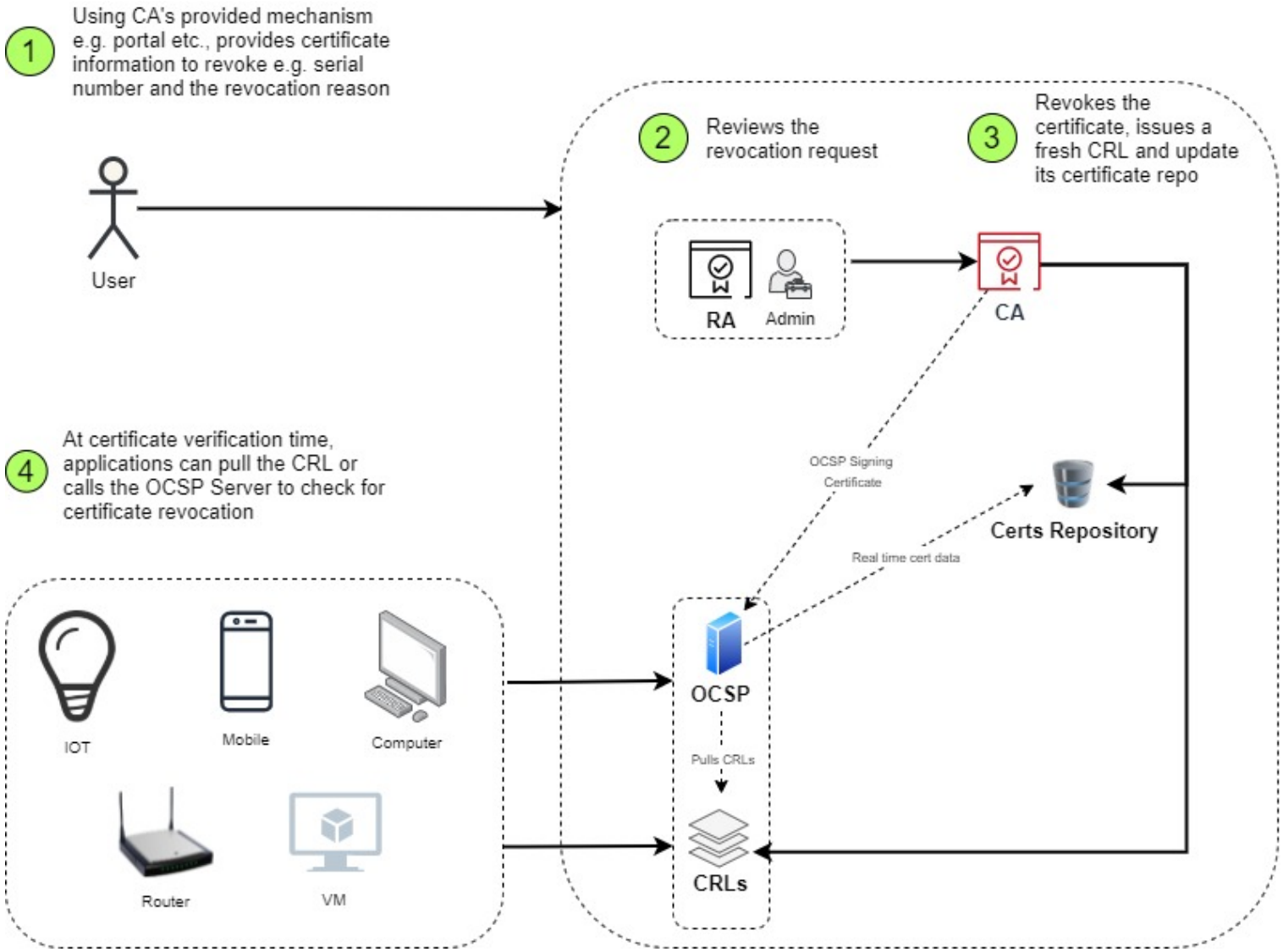
1. Hosts will submit certificate requests to the RA.
2. After the Registration Authority adds specific information to the certificate request and the request is approved under the organization's policy, it is forwarded on to the Certification Authority.
3. The CA will sign the certificate request and send it back to the host.

PKI Authentication Process



- https://www.geeksforgeeks.org/how-does-certificate-based-authentication-work/

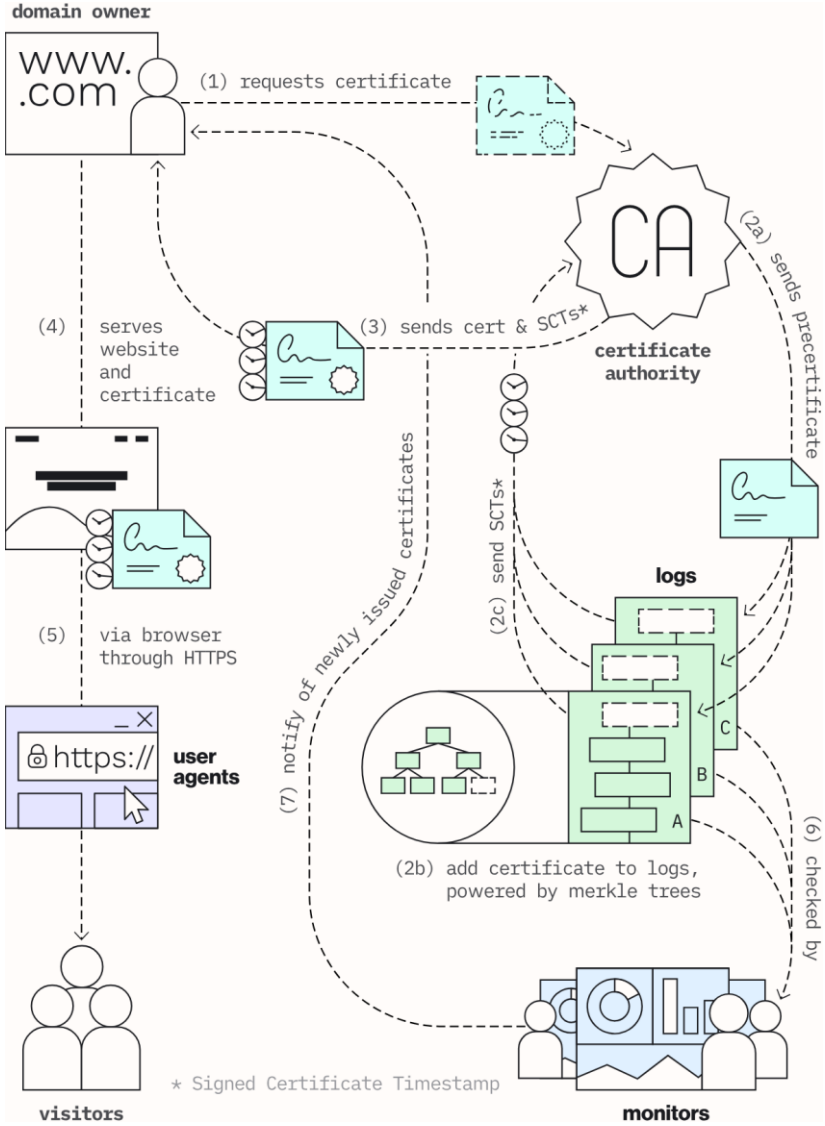
PKI Revocation Process



Certificates Transparency

- PKI is public, hence, can be monitored by anyone

- Monitor examples
  - https://certificate.transparency.dev/monitors/
  - https://crt.sh/?id=6106822638



- https://www.certkit.io/tools/ct-logs/?query=uba.ar

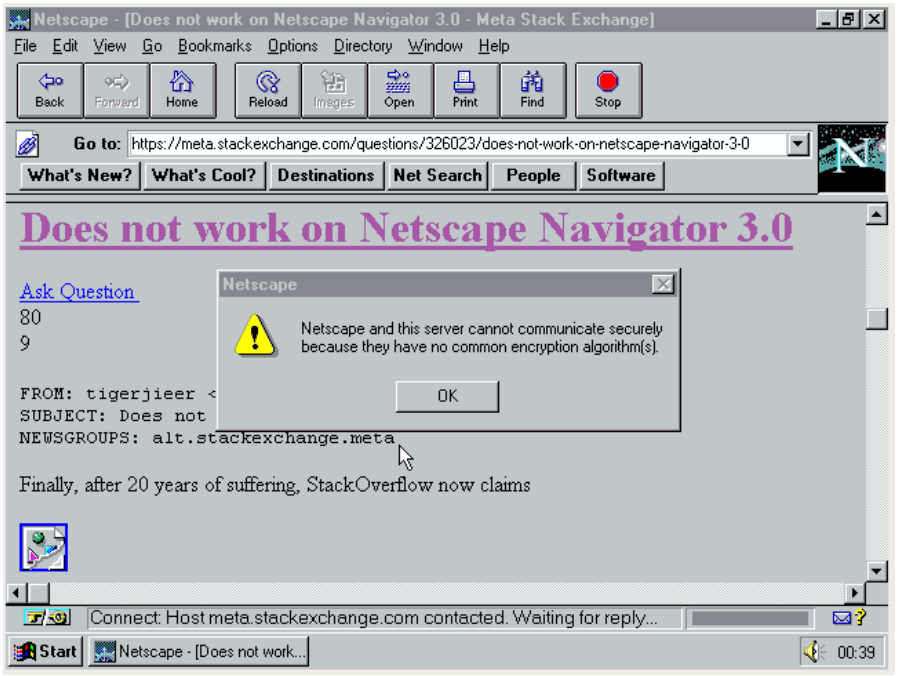
Results Found **12,374** total certificates. Showing the first **100**.

Not Before ▲▼	Days Remaining ▼	Common Name ▲▼	Matching SANs	Issuer ▲▼
2/26/2026	214 days	*.basesbiblioteca.derecho.uba.ar	*.basesbiblioteca.derecho.uba.ar basesbiblioteca.derecho.uba.ar	Sectigo Limited
12/18/2025	175 days	publicacionescientificas.fadu.uba.ar	publicacionescientificas.fadu.uba.ar www.publicacionescientificas.fadu.uba.ar	DigiCert Inc
5/20/2026	130 days	*.dc.uba.ar	*.dc.uba.ar	DigiCert Inc
5/4/2026	114 days	servicios.econ.uba.ar	contenidos.econ.uba.ar intranet.fce.uba.ar mail.fce.uba.ar mi.econ.uba.ar servicios.econ.uba.ar	Sectigo Limited
7/28/2026	90 days	saemcaem.qo.fcen.uba.ar	saemcaem.qo.fcen.uba.ar	Let's Encrypt
7/28/2026	90 days	www.sop.plataformalabrem.fi.uba.ar	www.sop.plataformalabrem.fi.uba.ar	Google Trust Services
7/28/2026	90 days	guarani2023-testing-uti.agro.uba.ar	guarani2023-testing-uti.agro.uba.ar	Let's Encrypt
7/28/2026	90 days	dev.sitios.fvet.uba.ar	dev.sitios.fvet.uba.ar	Let's Encrypt
7/28/2026	90 days	quicc.dc.uba.ar	quicc.dc.uba.ar	Let's Encrypt
7/28/2026	90 days	rspamd1.exactas.uba.ar	rspamd1.exactas.uba.ar	Let's Encrypt
7/28/2026	90 days	ventanillavirtual.rec.uba.ar	ventanillavirtual.rec.uba.ar	Let's Encrypt
7/28/2026	90 days	sietgraduados.rec.uba.ar	sietgraduados.rec.uba.ar	Let's Encrypt
7/28/2026	90 days	prismasubsidiostestext.rec.uba.ar	prismasubsidiostestext.rec.uba.ar	Let's Encrypt
7/28/2026	90 days	portal.dosuba.uba.ar	portal.dosuba.uba.ar	Let's Encrypt

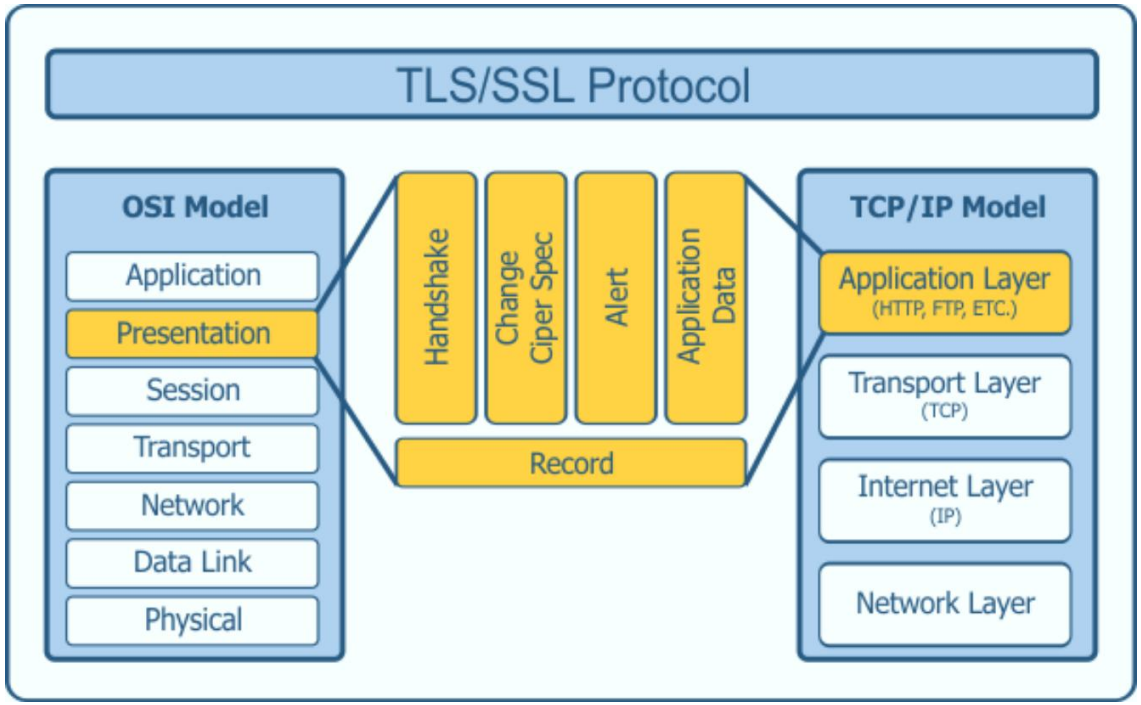
How is internet traffic secured nowadays?

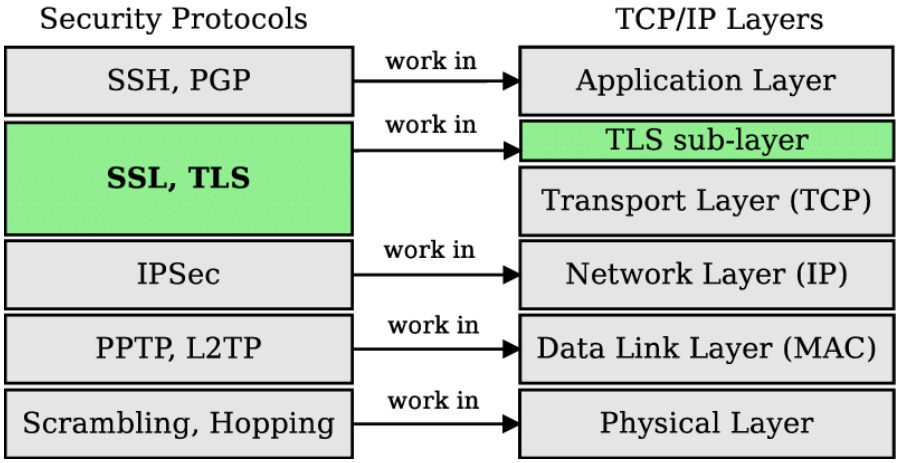
TRANSPORT LAYER SECURITY

- **TLS/SSL** are cryptographic protocols for secure communication.
  - SSL (Secure Sockets Layer): Introduced by Netscape in 1995.
  - TLS (Transport Layer Security): Successor to SSL, standardized by IETF.
- *Purpose = good old CIA*

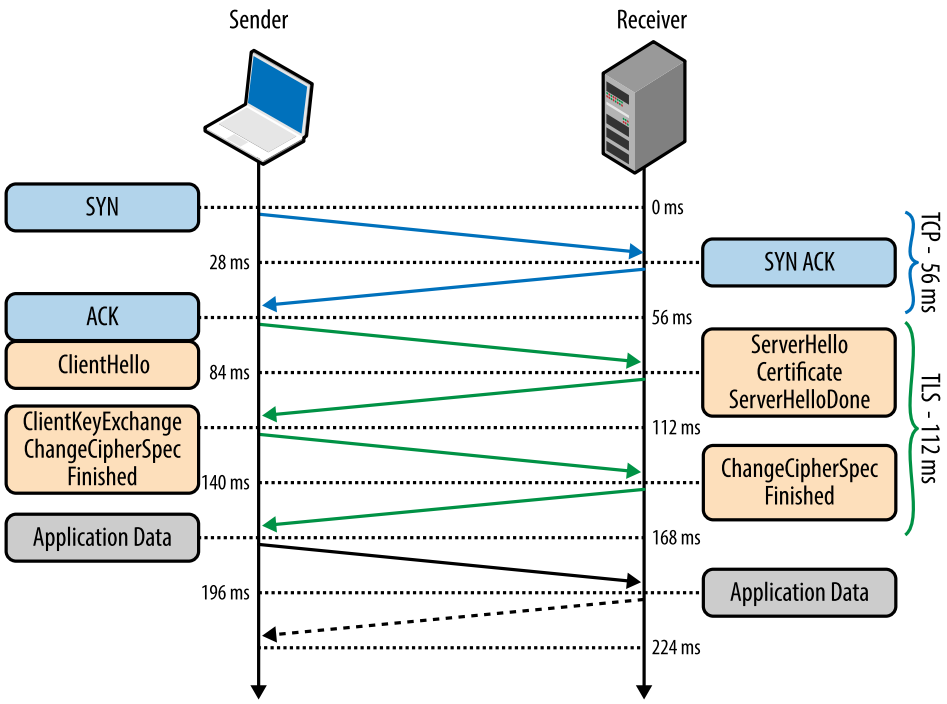


TLS/SSL in Stack





- **Phase 1:** TLS Handshake (Asymmetric Encryption)
  - Establishes a secure session and verifies identities via certificates
  - Negotiates encryption algorithms and keys
- **Phase 2:** Data Transmission (Symmetric Encryption)
  - Encrypts all data using a shared session key.
- Result: Secure communication with CIA



TLS/SSL Communication (2)

Message	Purpose	Contents
ClientHello	Initiates the handshake and proposes security settings.	• TLS Version • Random Number • Session ID (optional) • Cipher Suites • Extensions (SNI, ALPN, Supported Groups, Signature Algorithms)
ServerHello	Confirms security settings and provides server identity.	• TLS Version • Random Number • Selected Cipher Suite • Extensions (e.g., ALPN)
Server Certificate	Provides the server's identity for authentication.	• Digital Certificate (signed by CA) • Public Key • Certificate Chain (with intermediates) • CA Signature
Certificate Request (Optional)	Requests the client's certificate for mutual authentication.	• Certificate Types • CA Names (trusted authorities)
Client Certificate (Optional)	Provides the client's identity if requested by the server.	• Client Certificate • Public Key • Certificate Chain
Key Exchange (Pre-Master Secret)	Establishes a shared secret for session encryption.	• ECDHE Key Exchange (TLS 1.3) • Pre-Master Secret (Encrypted with Server's Public Key in TLS 1.2)
Certificate Verify (Optional)	Verifies the client's possession of the private key matching its certificate.	• Digital Signature (Signed hash of all handshake messages)
Finished Messages	Completes the handshake and verifies its integrity.	• Handshake Hash (Encrypted with session key) • Verifies handshake integrity

Cipher Suits (1)

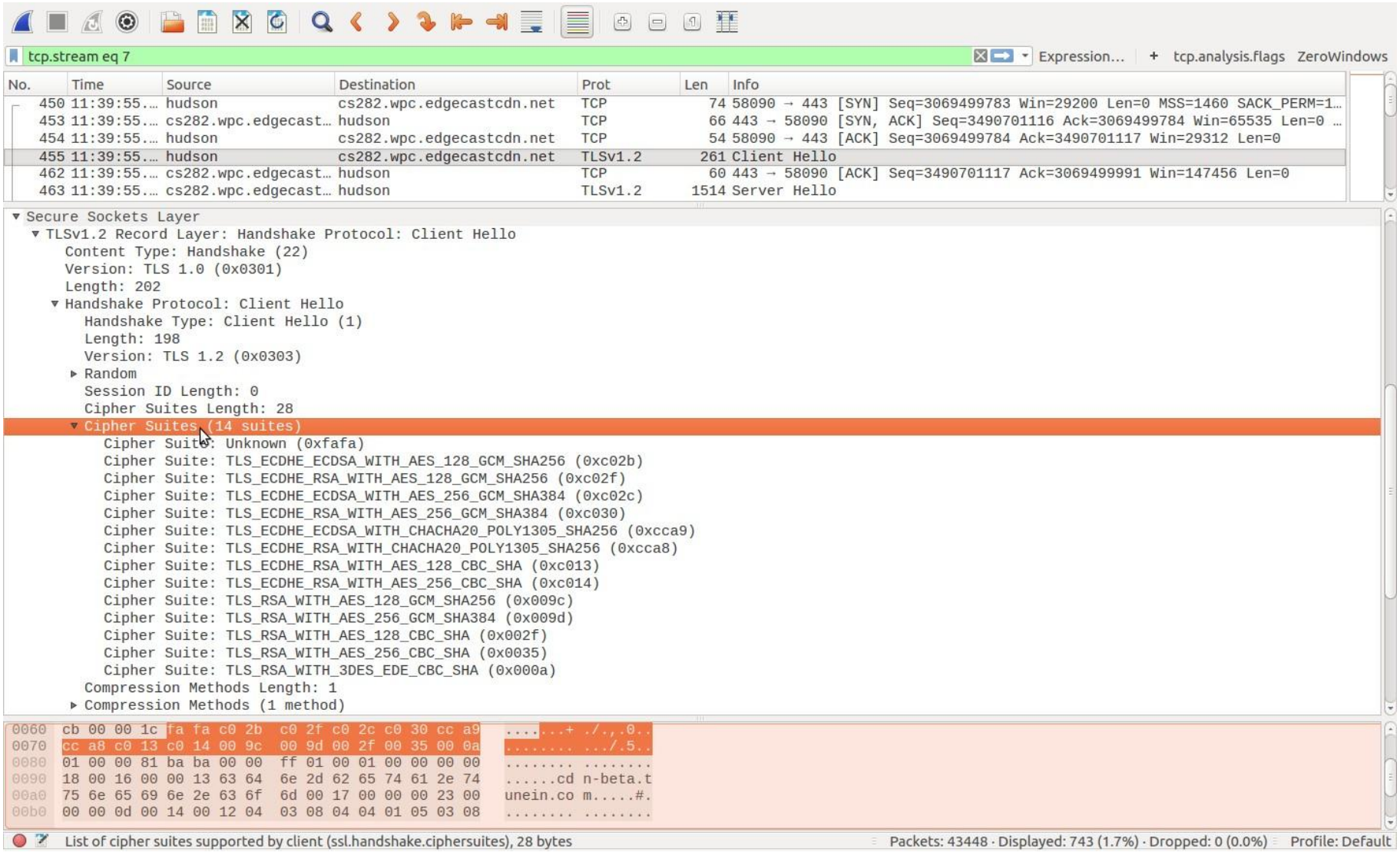
- TLS Cipher Suite Structure Example:

`TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`

  - TLS: Protocol
  - ECDHE: Key Exchange (Elliptic Curve Diffie-Hellman)
  - RSA: Authenticationz Algorithm
  - AES_128_GCM: Symmetric Encryption Algorithm
  - SHA256: Hashing Algorithm

- TLS improvements
  - only supports strong cipher suites.
  - removes outdated algorithms (e.g., RSA key exchange)

Cipher Suits (2)



History

Year	Version	Key Features and Changes
1995	SSL 2.0	• First publicly released version by Netscape. • Vulnerable to various attacks (e.g., handshake weaknesses). • Deprecated in 2011.
1996	SSL 3.0	• Major improvements over SSL 2.0. • Introduced support for MAC (Message Authentication Codes). • Vulnerable to POODLE attack. • Deprecated in 2015.
1999	TLS 1.0 (RFC 2246)	• First version of TLS, replacing SSL 3.0. • Added HMAC for stronger integrity checks. • Vulnerable to BEAST attack. • Deprecated in 2021.
2006	TLS 1.1 (RFC 4346)	• Added protection against CBC (Cipher Block Chaining) attacks. • Improved handling of padding errors. • Deprecated in 2021.
2008	TLS 1.2 (RFC 5246)	• Support for SHA-2 hashing algorithms. • Allowed authenticated encryption with AES-GCM. • Made cipher suites more configurable. • Still widely used today.
2018	TLS 1.3 (RFC 8446)	• Major security improvements and performance enhancements. • Removed support for outdated algorithms (e.g., RSA key exchange, CBC mode). • Faster handshakes with 1-RTT (one round-trip time). • Forward secrecy is mandatory. • Zero Round Trip Time (0-RTT) for session resumption. • Stronger default cipher suites (e.g., AES-GCM, ChaCha20-Poly1305).

https://www.ssldragon.com/blog/history-of-ssl-tls-versions/

- Common Threats
  - Expired Certificates: Cause service disruptions and warnings.
  - Man-in-the-Middle (MITM) Attacks: Interception of communication.
  - Cipher Suite Downgrade Attacks: Force use of weaker encryption.
  - Certificate Misuse: Stolen or fake certificates.
- Preventative Measures
  - Use HSTS (HTTP Strict Transport Security).
  - Enable `TLS_FALLBACK_SCSV` to prevent downgrades.
  - Implement Certificate Transparency Logs.
  - Use OCSP for real-time certificate status checking.

TLS Certificate for HTTP Server

```
<IfModule mod_ssl.c>
        <VirtualHost adresse_ip_du_site:443>

                ServerAdmin webmaster@localhost
                ServerName exemple.com
                ServerAlias www.exemple.com
                DocumentRoot /var/www/html

                Error
                Custo

                SSLEn

                SSLCe

                SSLCe

                SSLCe

                <File

                </Fil
                <Dire

                </Dir
        </VirtualHost
</IfModule>
```

```
server {

  # Listen on port 443
  listen   443 default_server;
  server_name example.com;

  root /path/to/site-content/;
  index index.html index.htm;

  # Turn on SSL; Specify certificate & keys
  ssl on;
  ssl_certificate /etc/nginx/ssl/example.com/my_certificate.crt;
  ssl_certificate_key /etc/nginx/ssl/example.com/example.key;

  # Enable OCSP Stapling, point to certificate chain
  ssl_stapling on;
  ssl_stapling_verify on;
  ssl_trusted_certificate /etc/nginx/ssl/full_chain.pem;

 }
```

TLS Monitoring

- https://www.ssllabs.com/ssl-pulse/
- https://www.ssllabs.com/ssltest/analyze.html?d=www.fit.vutbr.cz&s=147.229.9.23&latest

Strict Transport Security

50,225 Sites that support **HTTP Strict Transport Security**

37.4 % of sites surveyed

- 6 since previous month

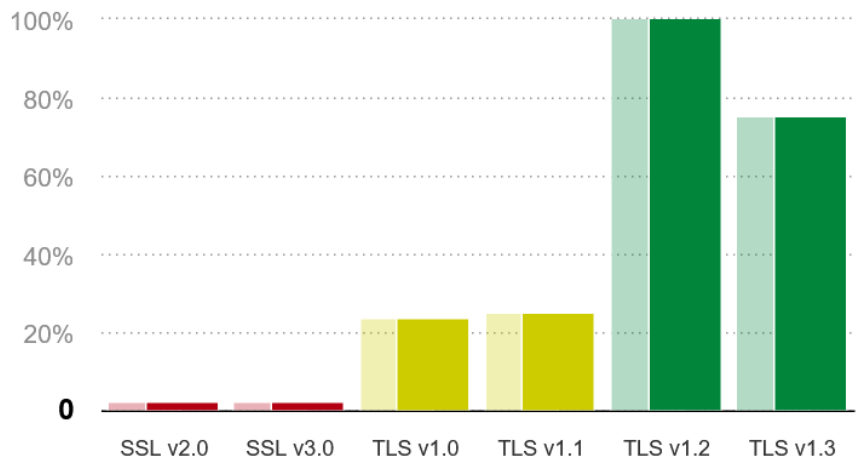
CAA

22,461 Sites that support **Certification Authority Authorization** (RFC 6844)

16.7 % of sites surveyed

- 2 since previous month

Protocol Support



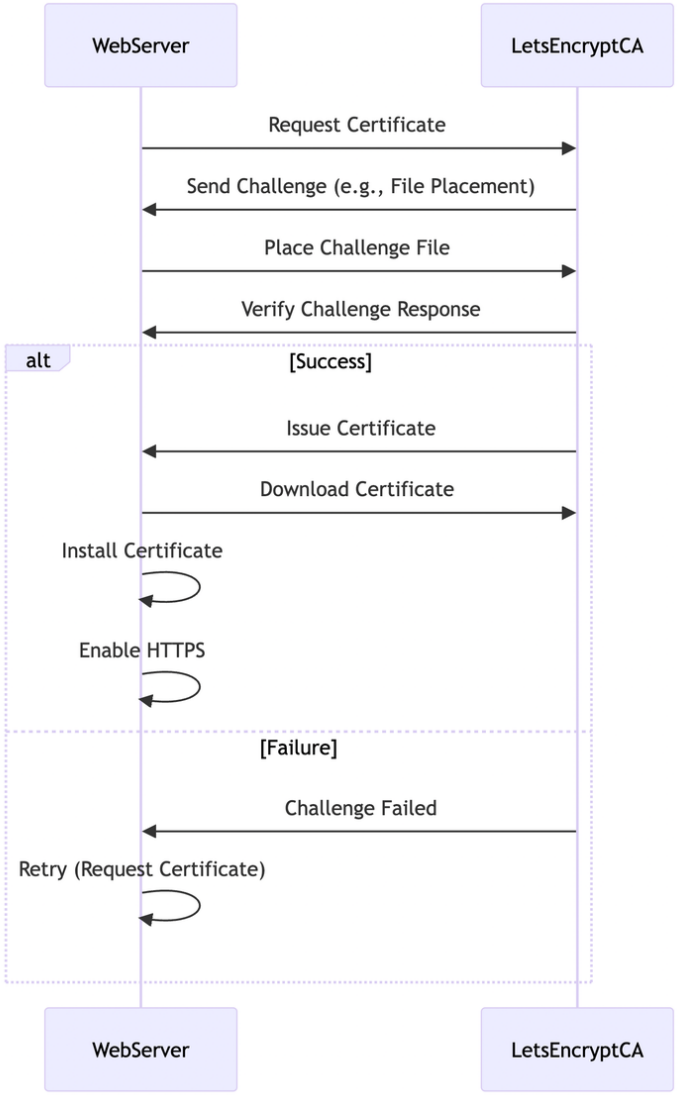
Let's Encrypt

- Let's Encrypt is a **free, automated, and open** CA
- Launched in December 2015 by the Internet Security Research Group (ISRG)
  - https://www.abetterinternet.org/
- Provides SSL/TLS certificates to enable HTTPS and secure communication.
- Mission: *"Encrypt the entire web" to promote a safer and more privacy-respecting internet*.
  - Helps website owners secure their websites without cost or complexity.
  - Supported by: Major sponsors including Mozilla, Cisco, Google Chrome, Akamai, and Electronic Frontier Foundation (EFF).

How LE Works?

1) **Domain Validation (DV)**
   - Let's Encrypt provides DV certificates, verifying domain ownership through:
     - HTTP Challenge: Places a unique token on the website.
     - DNS Challenge: Adds a specific DNS TXT record.
2) **ACME Protocol** (Automatic Certificate Management Environment)
   - ACME is an open protocol that automates the certificate issuance and renewal process.
   - Users interact with ACME through ACME clients (e.g., Certbot).
3) **Certificate Issuance**
   - Certificates are issued with a 90-day validity period for increased security.
   - Can include wildcard certificates (e.g., *.example.com) using DNS validation.
4) **Auto-Renewal**
   - Automatic renewal is achieved using ACME clients with cron jobs or systemd timers.
   - ACME clients request a new certificate before expiration and update the web server automatically.

LE: Demonstration



https://community.f5.com/kb/technicalarticles/lets-encrypt/328574

ACME Clients

- **Certbot** *(Most Popular Client)*
  - developed by the Electronic Frontier Foundation (EFF).
  - supports automatic certificate installation for Apache, Nginx, etc.
- **acme.sh** *(CLI)*
  - a pure bash client with zero dependencies
  - ideal for custom deployments and Docker environments.
  - supports wildcard certificates with DNS validation.
- **Caddy** *(Web Server with Built-in HTTPS)*
  - A web server that automatically obtains and renews
  - requires zero configuration for HTTPS
- **Traefik** *(Reverse Proxy for Containers)*
  - ideal for microservices and Docker-based environments.
  - automatically handles Let's Encrypt integration and certificate management.
  - suitable for Kubernetes and cloud deployments.
- **Nginx**
  - Native support using https://nginx.org/en/docs/http/ngx_http_acme_module.html

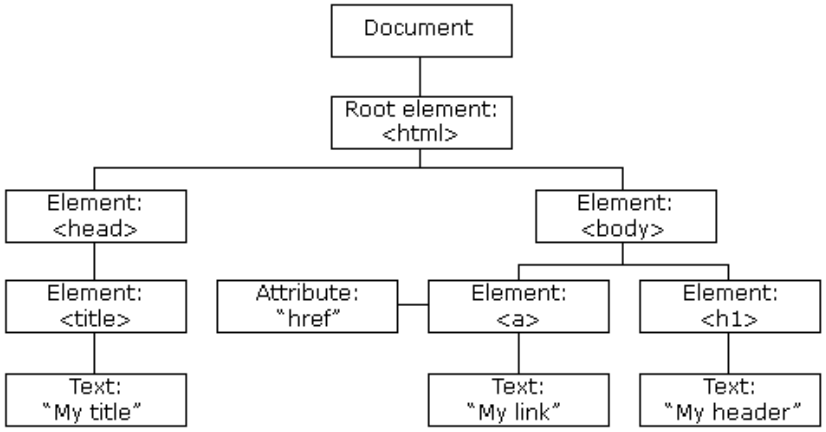
IDN Attacks

- https://en.wikipedia.org/wiki/IDN_homograph_attack
- https://www.xudongz.com/blog/2017/idn-phishing/

DOMAIN	DISPLAY FORM	PUNYCODE	VERDICT
paypal.com	paypal.com	paypal.com	Clean ASCII
münchen.de	münchen.de	xn--mnchen-3ya.de	Legitimate IDN — single script (Latin extended)
北京.中国	北京.中国	xn--1lq90ic7f.xn--fiqs8s	Legitimate IDN — Chinese script throughout
paypal.com	paypal.com	xn--ypal-2of2c.com	Homograph — mixes Cyrillic + Latin
paypal.com	paypal.com	xn--pypa-43dz3a.com	Homograph — single Cyrillic + Cherokee 'I'
g00gle.com	g00gle.com	g00gle.com	Typosquatting (digit-letter substitution; not Punycode)

HYPERTEXT TRANSFER PROTOCOL

- A web page contains objects
  - An object can be an HTML file, JPEG image, Java applet, audio file, ...
- A web page consists of a base HTML file, which contains links to other objects
- Each object is addressable using a URL, generally following a URI (which specifies access to the object ~ protocol

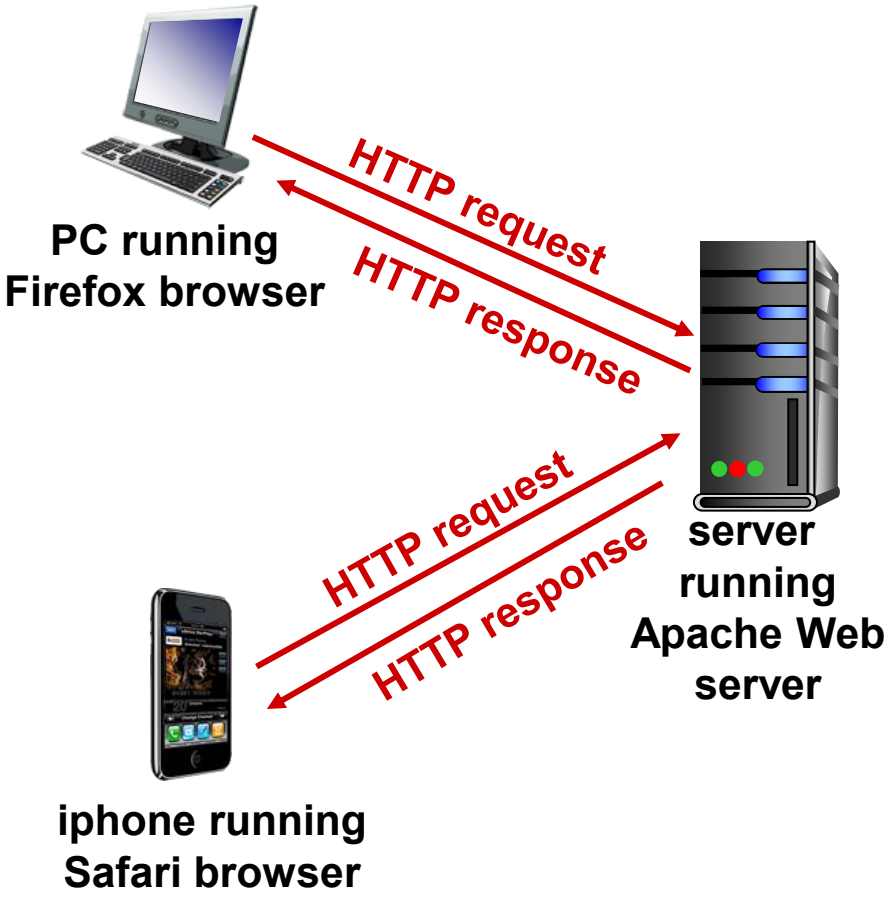


www.someschool.edu/someDept/pic.gif

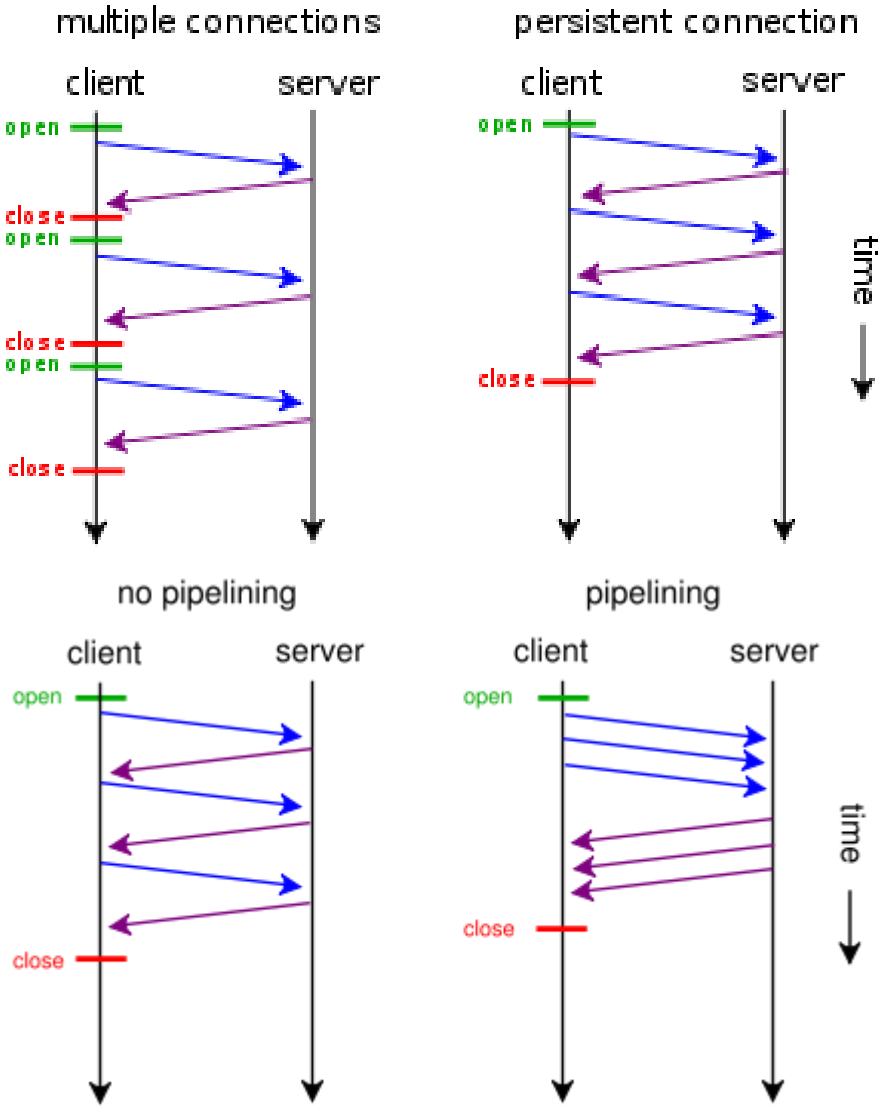
host name | path name

HyperText Transfer Protocol

- Web's application layer protocol
  - TCP over web
  - RFC 2616
- **Client**
  - browser communicating using HTTP
  - Initiates connection to server port 80
- **Server**
  - web server responding to client with web objects
  - listens on port 80/443 and accepts connection (it listen elsewhere as well)
- HTTP is stateless
  - server does not remember anything about previous client requests
  - simple request-response scheme

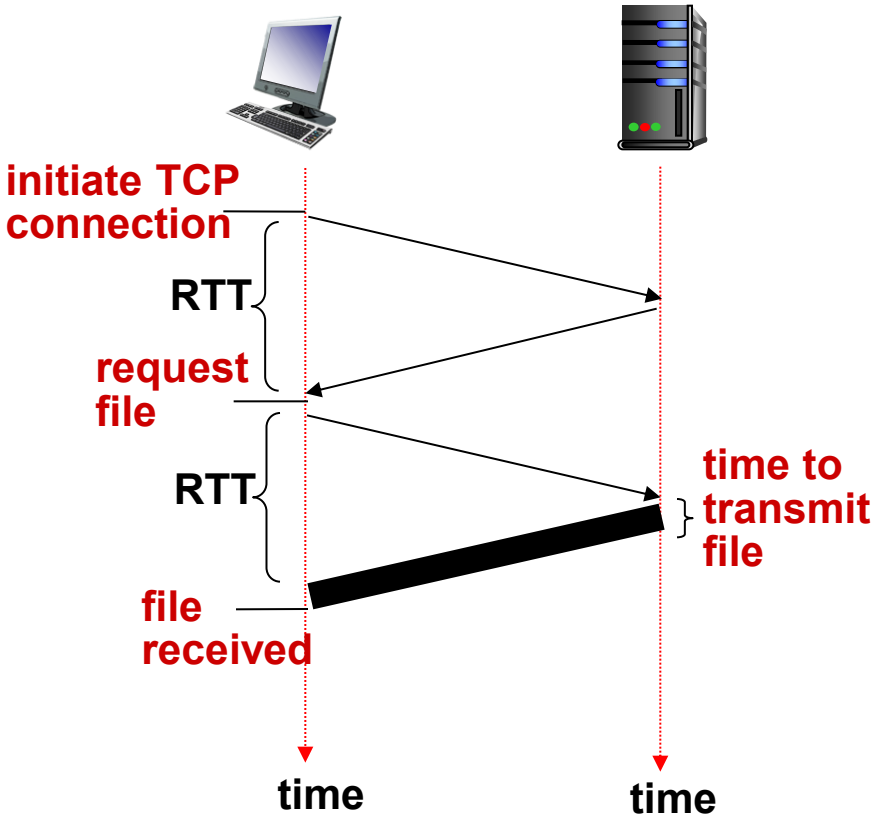


- **Non-persistent**
  - max. one web object at the time
  - HTTP/1.0
- **Persistent**
  - More object through single connection
  - HTTP/1.1
- **Pipelining**
  - Since HTTP/1.1

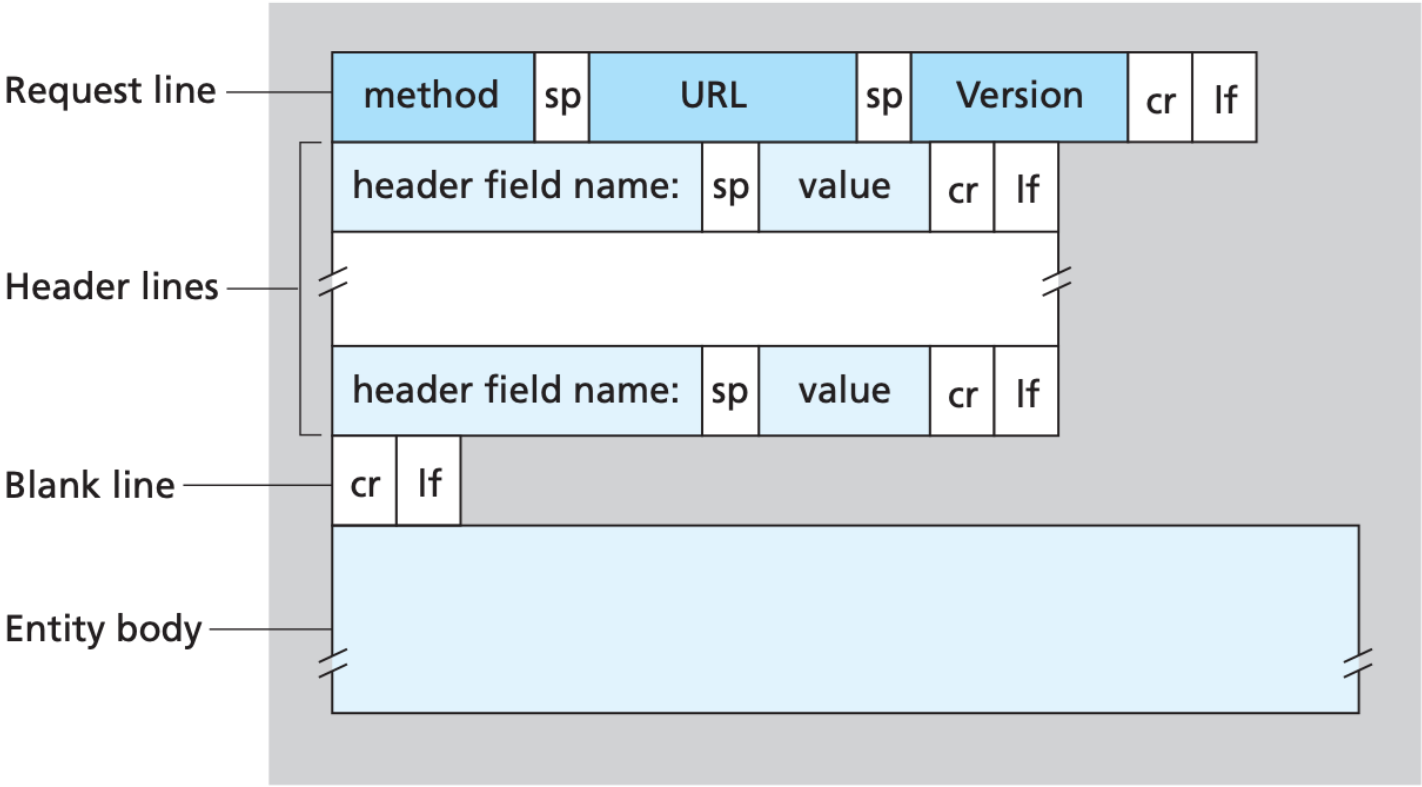


Non-persistent HTTP response time

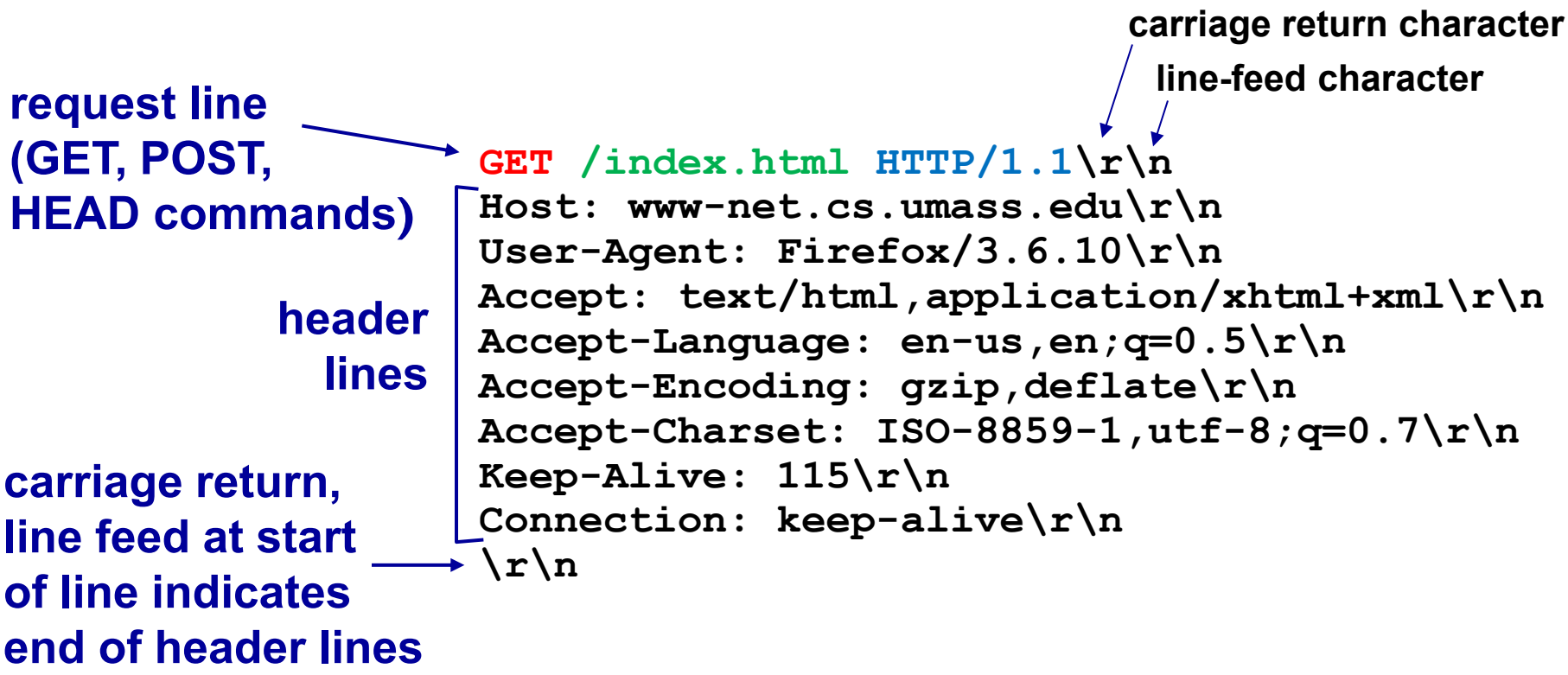
- Round-trip Time (RTT)
  - time for a small packet to travel from client to server and back
- HTTP response time
  - one RTT to initiate TCP connection
  - one RTT for HTTP request and first few bytes of HTTP response to return
  - file transmission time



- Two types of HTTP messages: request, response
- **HTTP Request 1.*** message:
  - ASCII (human-readable format)



HTTP 1.1 Request: Example

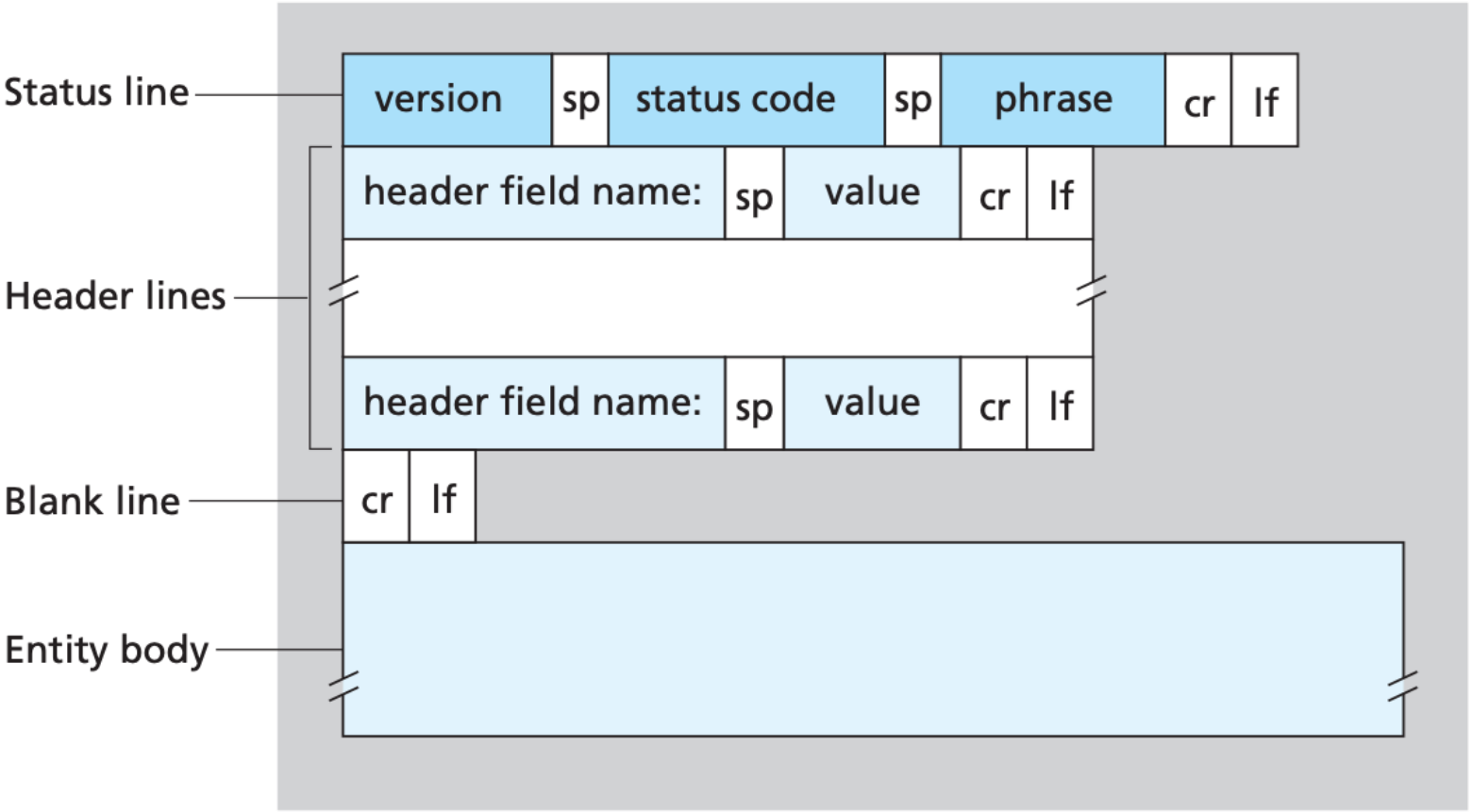


Methods

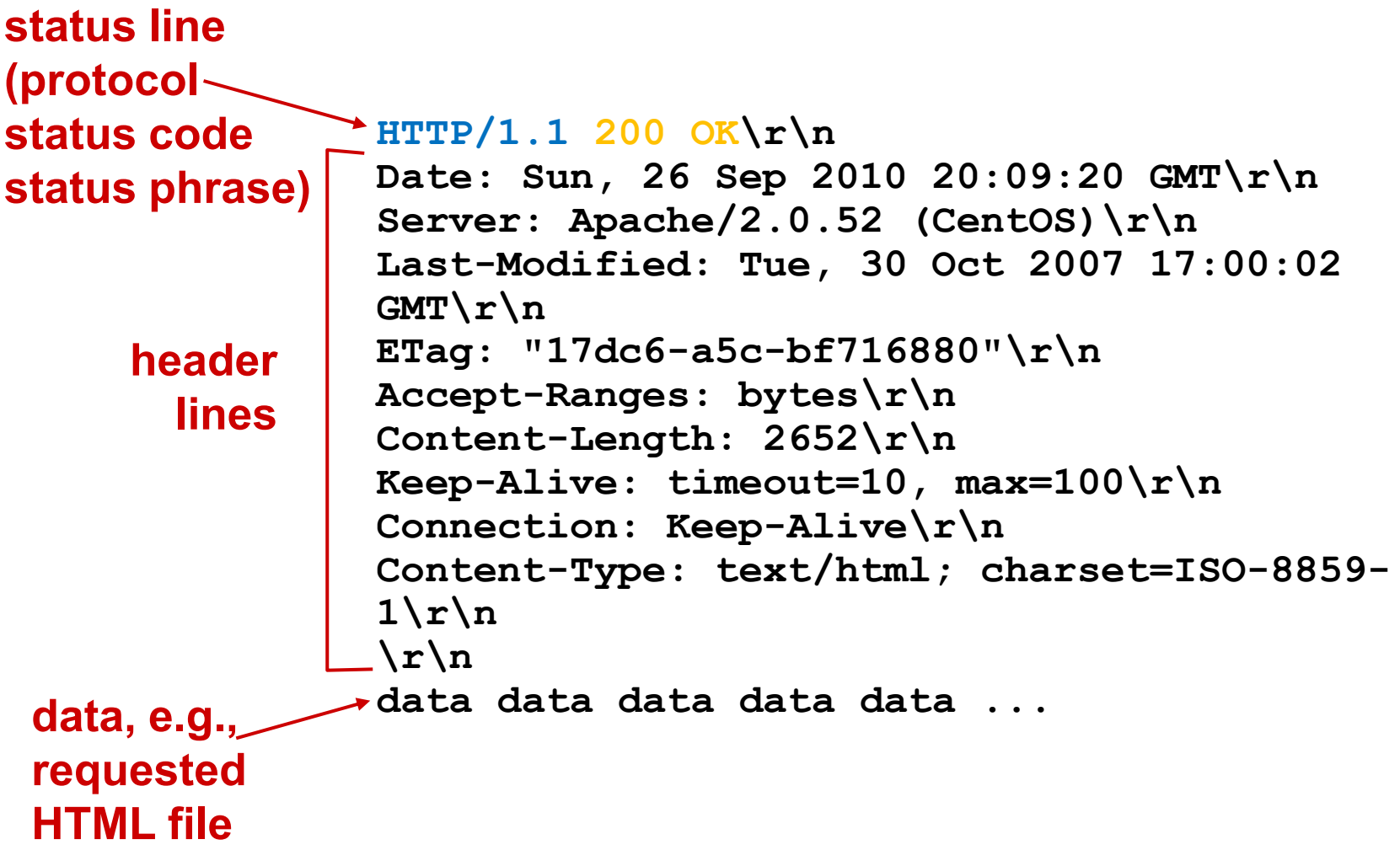
- **HTTP/1.0**
  - GET
  - POST
  - HEAD
- **HTTP/1.1**
  - GET, POST, HEAD
  - PUT
  - DELETE
  - Optional
    - LINK, UNLINK, PATCH
- **HTTP/2.0**
  - Google
  - SPDY protocol
- **HTTP/3.0**
  - QUIC

Operation	Description
OPTIONS	Request information about available options
GET	Retrieve document identified in URL
HEAD	Retrieve metainformation about document identified in URL
POST	Give information (e.g., annotation) to server
PUT	Store document under specified URL
DELETE	Delete specified URL
TRACE	Loopback request message
CONNECT	For use by proxies

- **HTTP Response 1.*** message format:



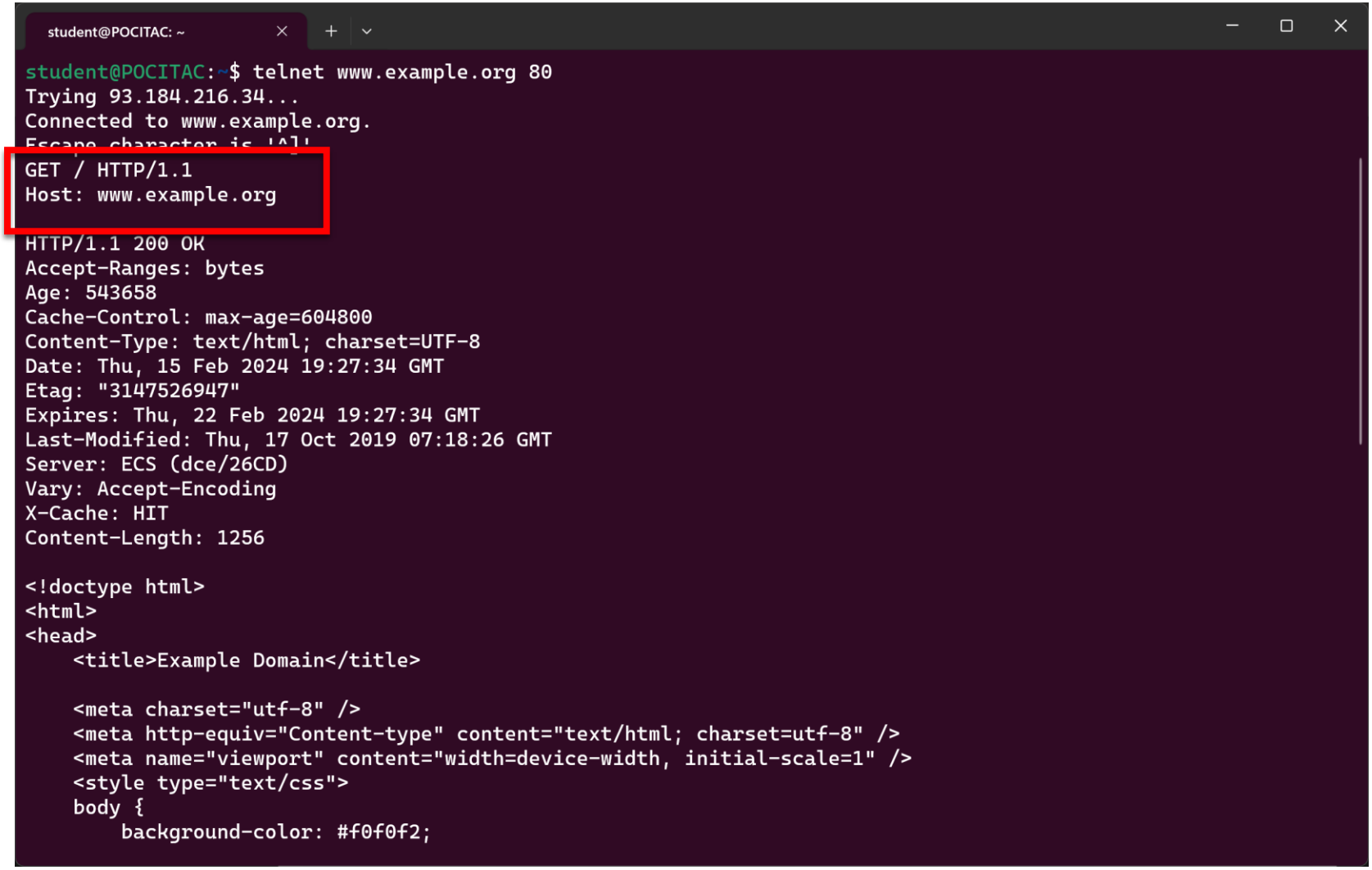
HTTP 1.1 Response: Example



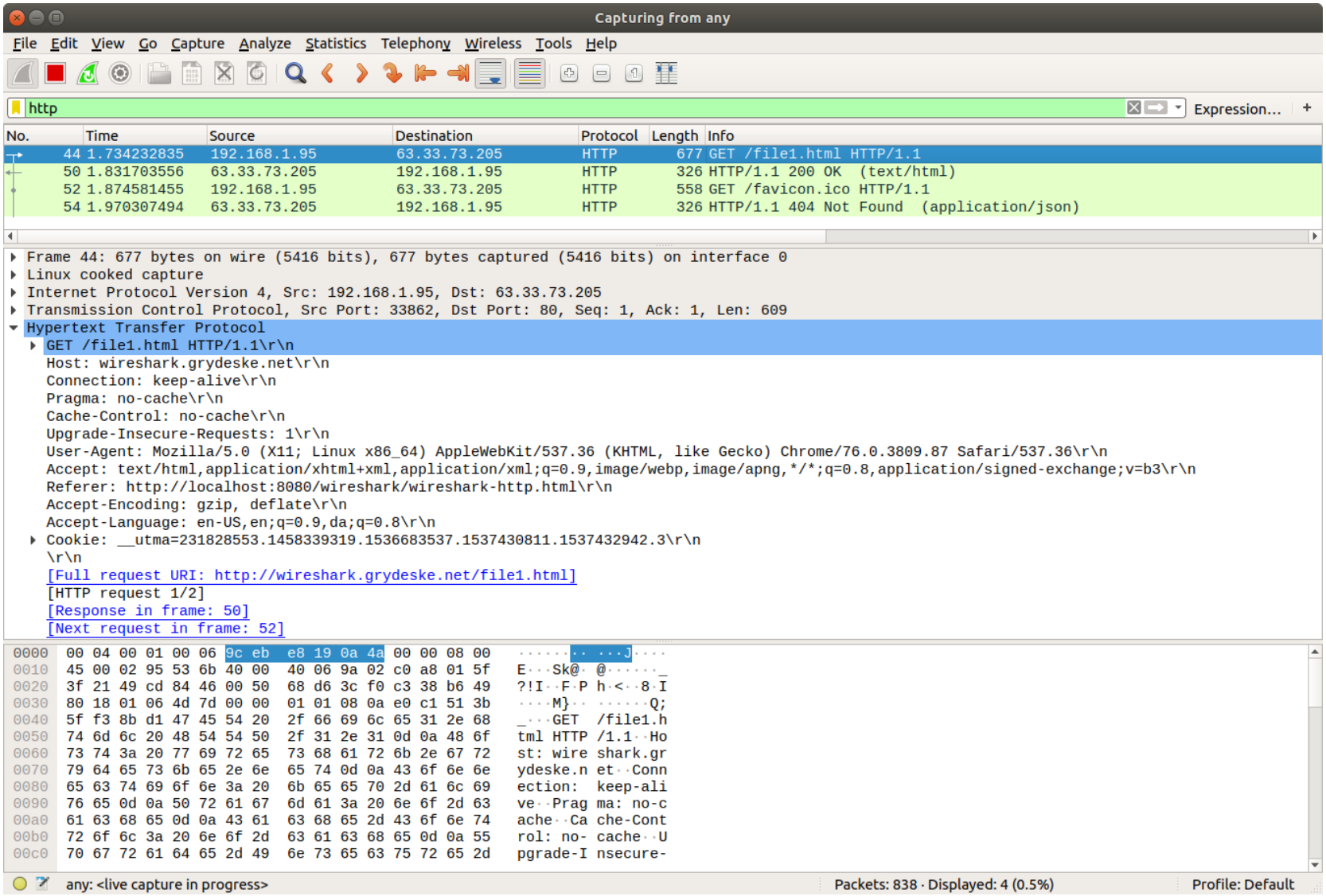
Response Codes

- status code appears in 1st line in server-to-client response message.
- some sample codes:
  - **200 OK**
    - request succeeded, requested object later in this msg
  - **301 Moved Permanently**
    - requested object moved, new location specified later in this msg (Location:)
  - **400 Bad Request**
    - request msg not understood by server
  - **404 Not Found**
    - requested document not found on this server
  - **418 I'm a teapot** (see https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/418)
  - **505 HTTP Version Not Supported**

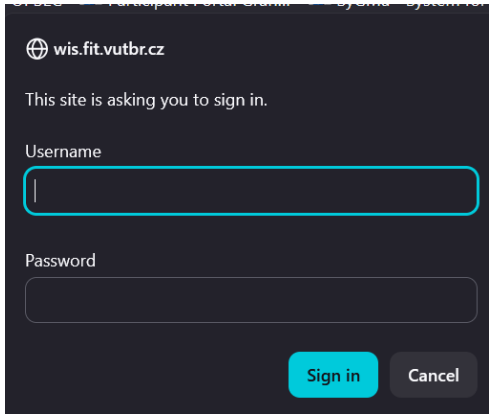
Homework #1


student@POCITAC: ~
student@POCITAC:~$ telnet www.example.org 80
Trying 93.184.216.34...
Connected to www.example.org.
Escape character is '^]'.
GET / HTTP/1.1
Host: www.example.org
HTTP/1.1 200 OK
Accept-Ranges: bytes
Age: 543658
Cache-Control: max-age=604800
Content-Type: text/html; charset=UTF-8
Date: Thu, 15 Feb 2024 19:27:34 GMT
Etag: "3147526947"
Expires: Thu, 22 Feb 2024 19:27:34 GMT
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT
Server: ECS (dce/26CD)
Vary: Accept-Encoding
X-Cache: HIT
Content-Length: 1256
<!doctype html>
<html>
<head>
<title>Example Domain</title>
<meta charset="utf-8" />
<meta http-equiv="Content-type" content="text/html; charset=utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<style type="text/css">
body {
background-color: #f0f0f2;

HTTP: Example

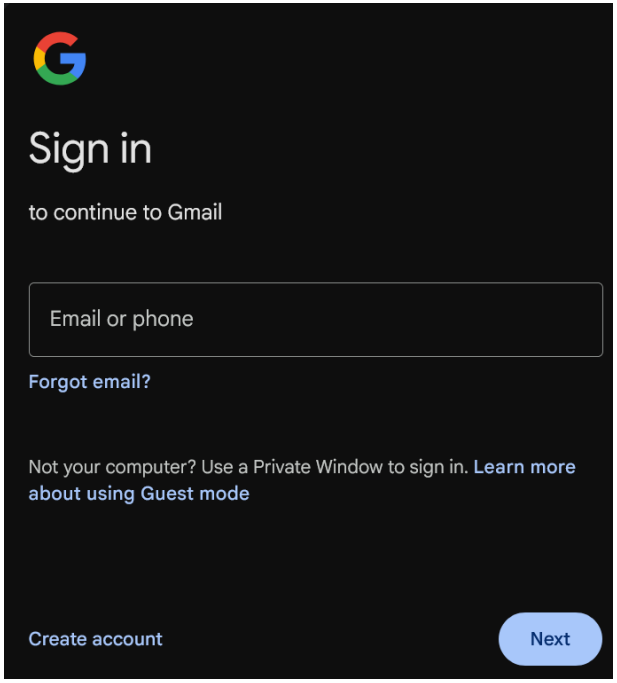


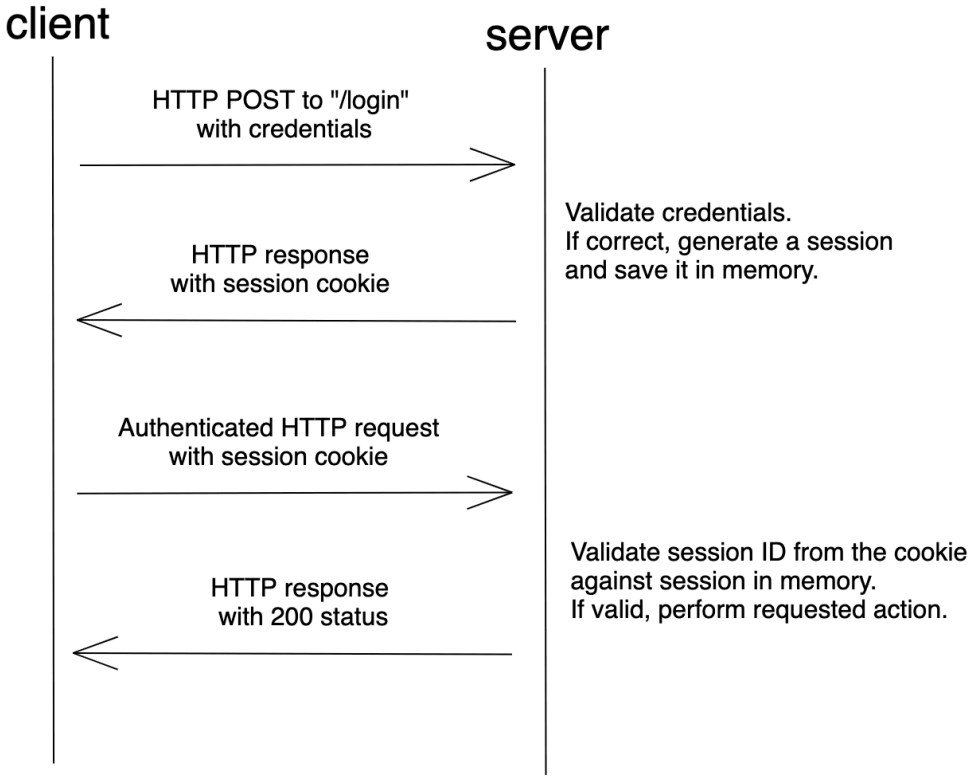
- Base64: YWxhZGRpbjpvcGVuc2VzYW11



HTTP Auth: Other Methods

- Session-based





- Token-based (JWT)
- OTPs
- OAuth2

AI Consideration

User:

Here you have request:

```
GET /FIT/db/vav/project.php HTTP/1.1
Host: wis.fit.vutbr.cz
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0) Gecko/20100101
Firefox/153.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: cz-CZ,cz;q=0.5
Accept-Encoding: gzip, deflate, br, zstd
Referer: https://wis.fit.vutbr.cz/FIT/db/vav/
Authorization: Basic dmasdVzZWx5dasfasfUADKncaskně==
Connection: keep-alive
Cookie: wislang=cs; SimpleSAML=097a987abbff700c97
Upgrade-Insecure-Requests: 1
```

Use it to access https://wis.fit.vutbr.cz/FIT/db/vav/project.php On this webpage parse column `Název` and I want you to match it with `Název` in `ksaft-search.csv`. The goal of parsing is to get project_id and pivot to project detail page, such as https://wis.fit.vutbr.cz/FIT/db/vav/project.php?id=2090

AI:

Security reminder (again, emphatically)

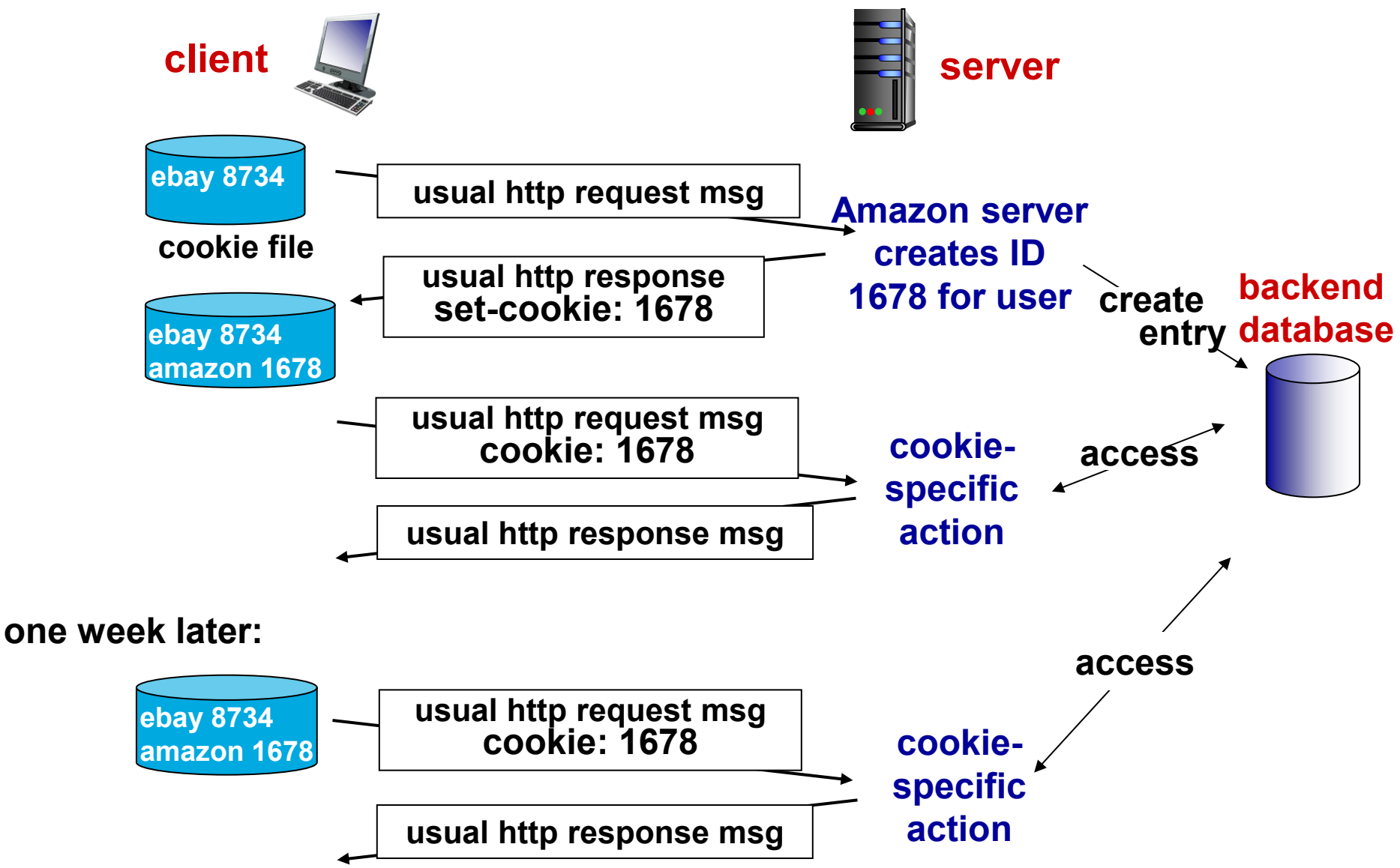
⚠️ The credentials from the header (`veselyv` + password) as well as the `SimpleSAML` session token are now in this conversation's history and in the intermediate outputs. The password in particular was transmitted in cleartext via HTTP Basic auth. I recommend changing it once you're done.

Cookies

- Web sites and client browser use cookies to maintain some state between transactions
- four components:
  1. cookie header line of HTTP Response message
  2. cookie header line in next HTTP Request message
  3. cookie file kept on user's host, managed by user's browser
  4. back-end database at Web site

- Example:
  - Susan uses browser on laptop, visits specific e-commerce site for first time
  - when initial HTTP requests arrives at site, site creates:
    - unique ID (aka "cookie")
    - entry in backend database for ID
  - subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to "identify" Susan

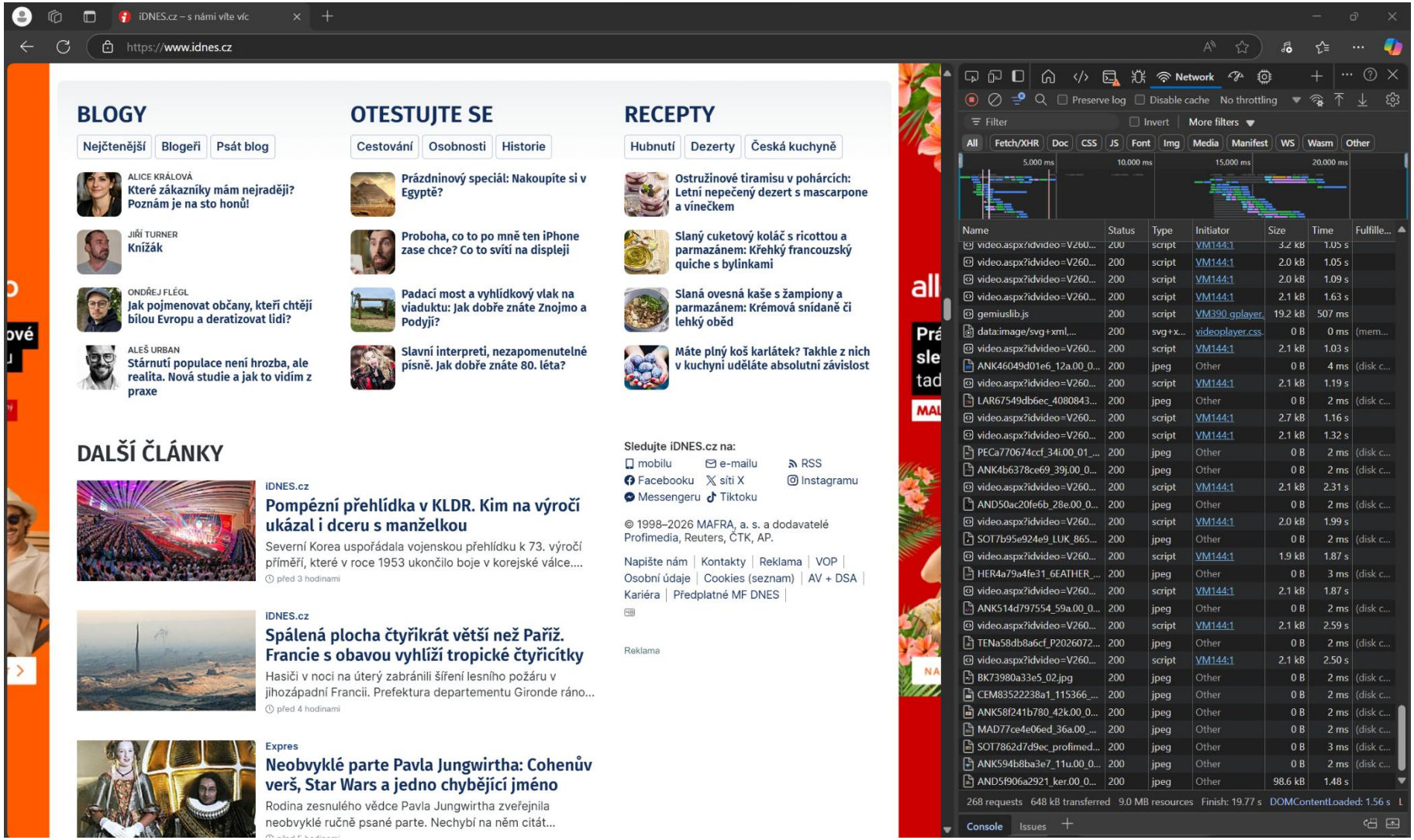
Cookies: keeping “state”(cont.)



Cookies (continued)

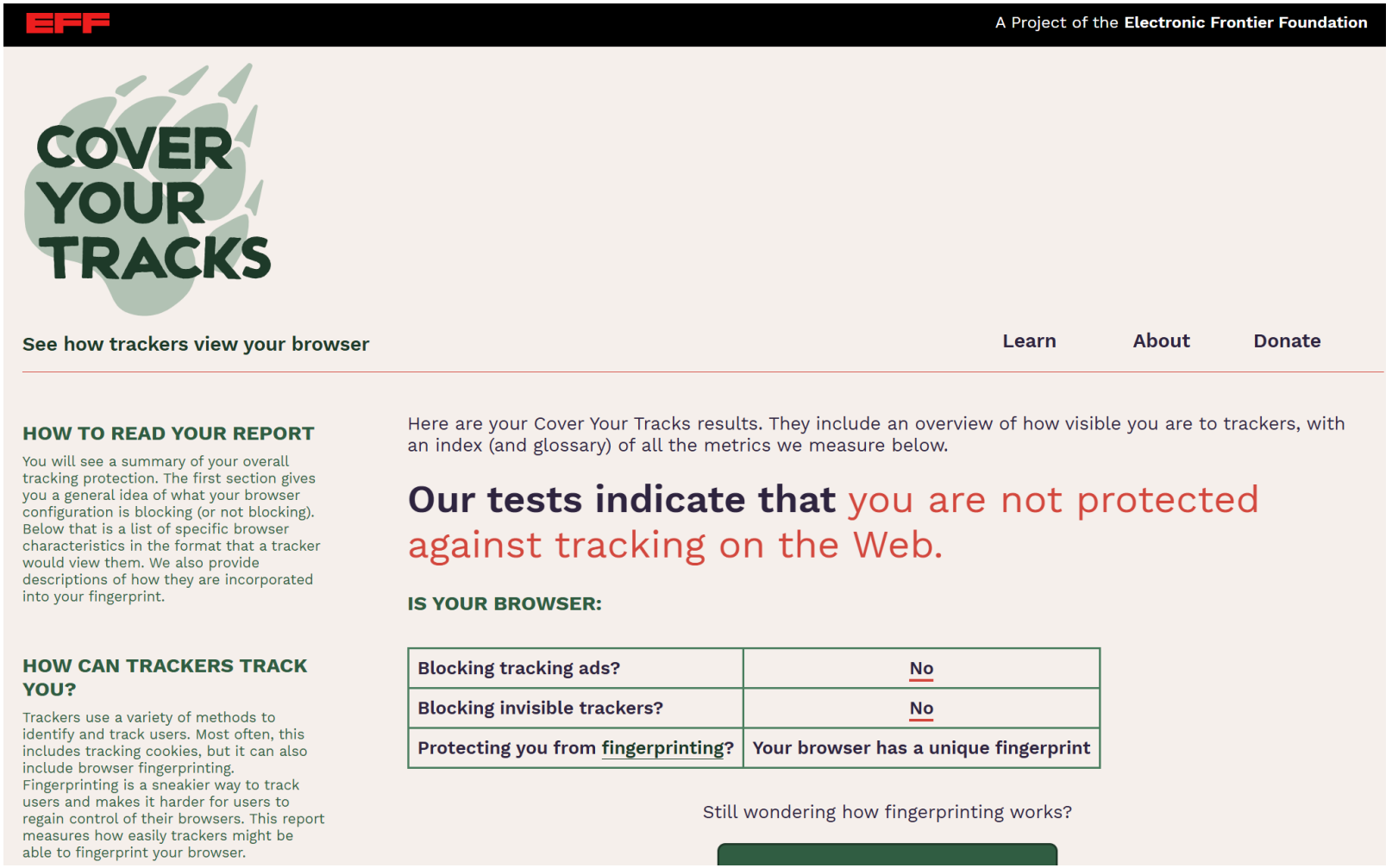
- What cookies can be used for:
  - authorization
  - shopping carts
  - recommendations
  - user session state (Web e-mail)
- How to keep “state”:
  - protocol endpoints: maintain state at sender/receiver over multiple transactions
  - cookies: http messages carry state
- Privacy
  - cookies permit sites to learn a lot about you
  - you may supply name and e-mail to sites

Fingerprinting via Cookies



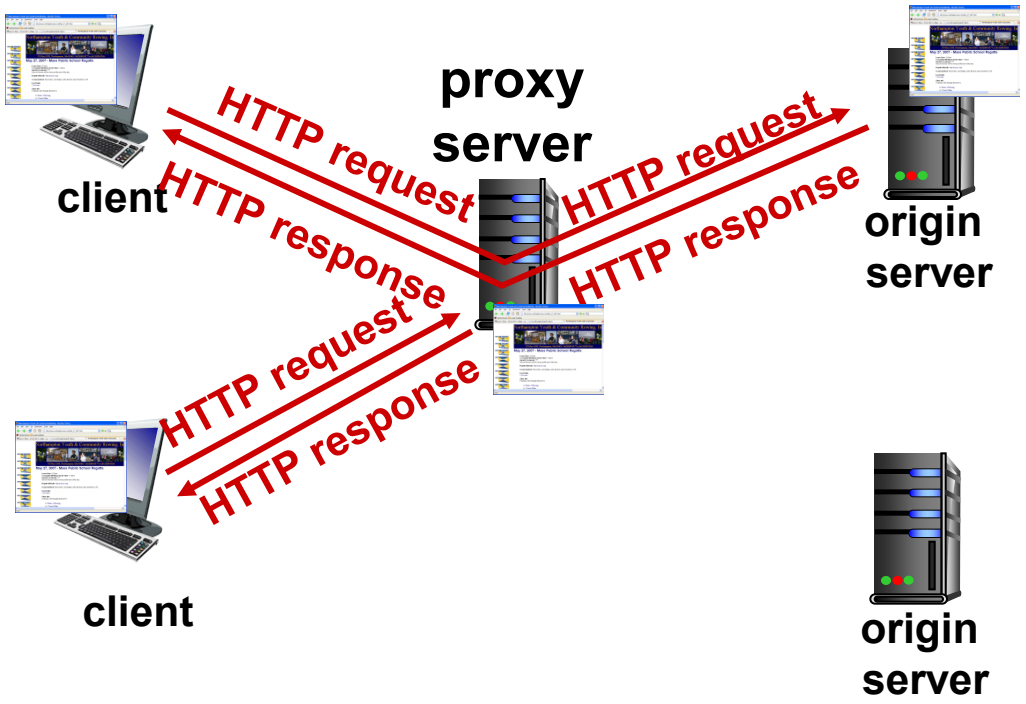
How unique is your browser?

- https://coveryourtracks.eff.org/



Web caches / Proxies (1)

- goal: satisfy client request without involving origin server
- user sets browser: Web accesses via cache
- browser sends all HTTP requests to cache
  - object in cache: cache returns object
  - else cache requests object from origin server, then returns object to client



Web caches / Proxies (2)

- Web cache acts as both client and server
  - server for original requesting client
  - client to origin server
- server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

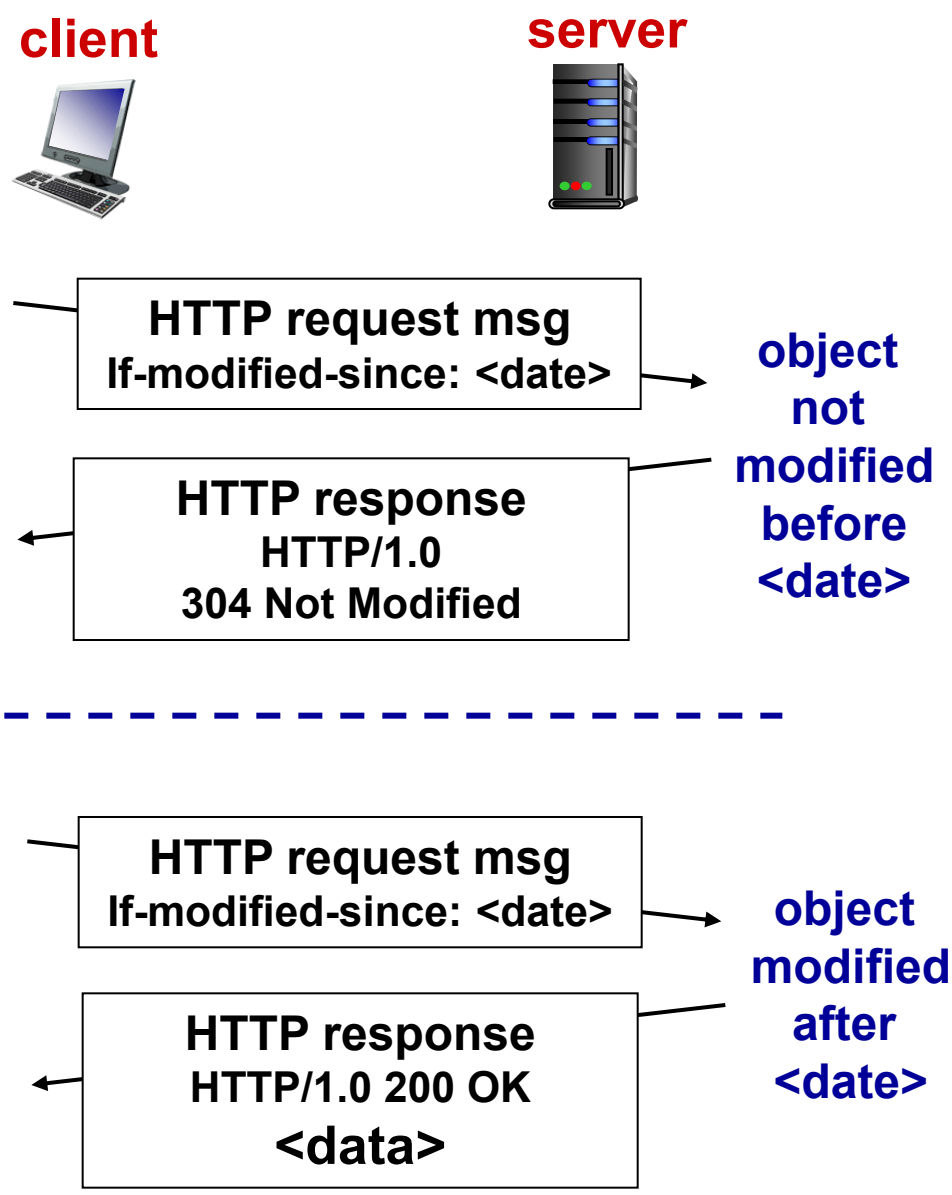
```
Cache-Control: no-cache
```

Why Web caching?

- reduce response time for client request
  - cache is closer to client
- reduce traffic on an institution's access link
- Internet is dense with caches
  - enables "poor" content providers to more effectively deliver content

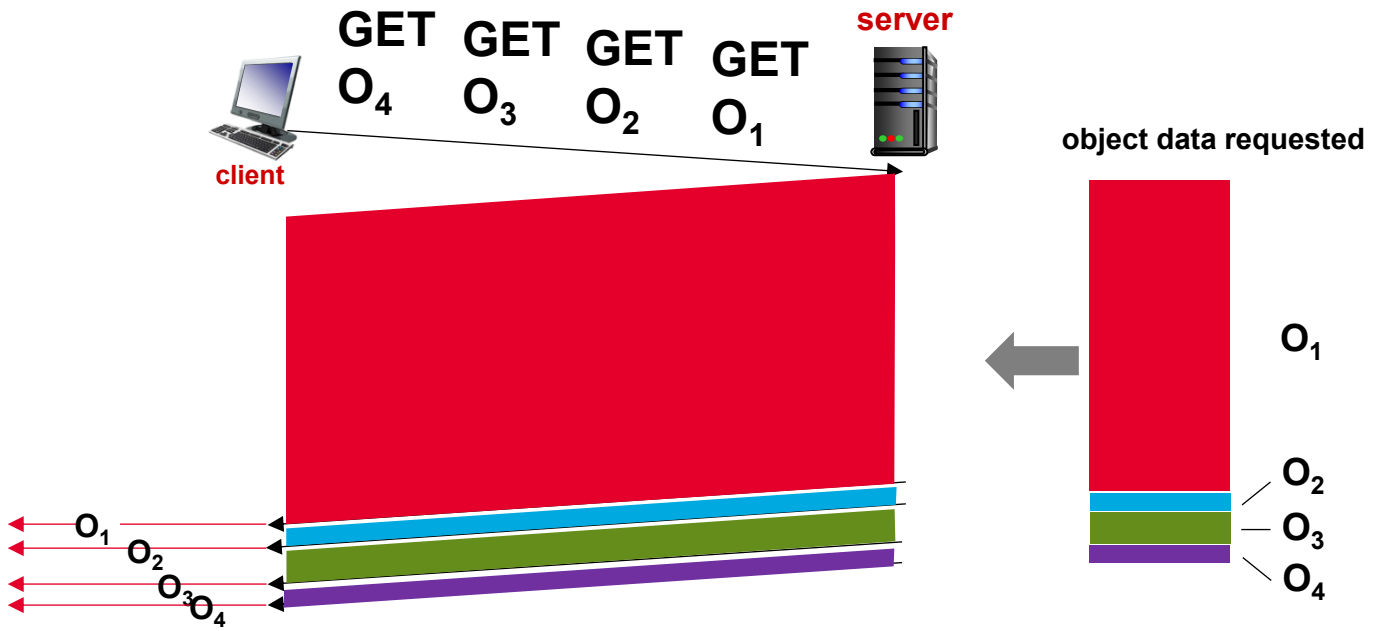
Conditional GET

- Goal: don't send object if cache has up-to-date cached version
  - no object transmission delay
  - lower link utilization
- cache: specify date of cached copy in HTTP request
  - `If-modified-since: <date>`
- server: response contains no object if cached copy is up-to-date:
  - HTTP/1.0 **304 Not Modified**



HTTP/2: Mitigating HOL blocking

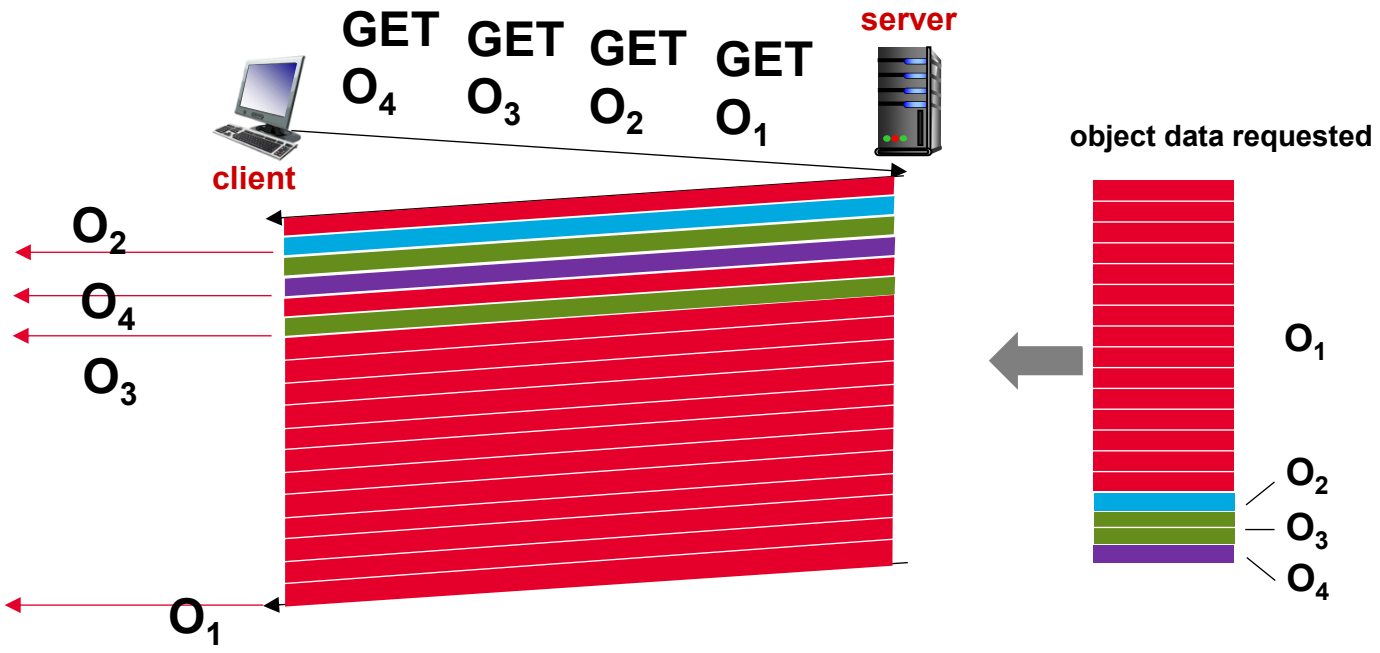
- HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects



objects delivered in order requested: O_2, O_3, O_4 wait behind O_1

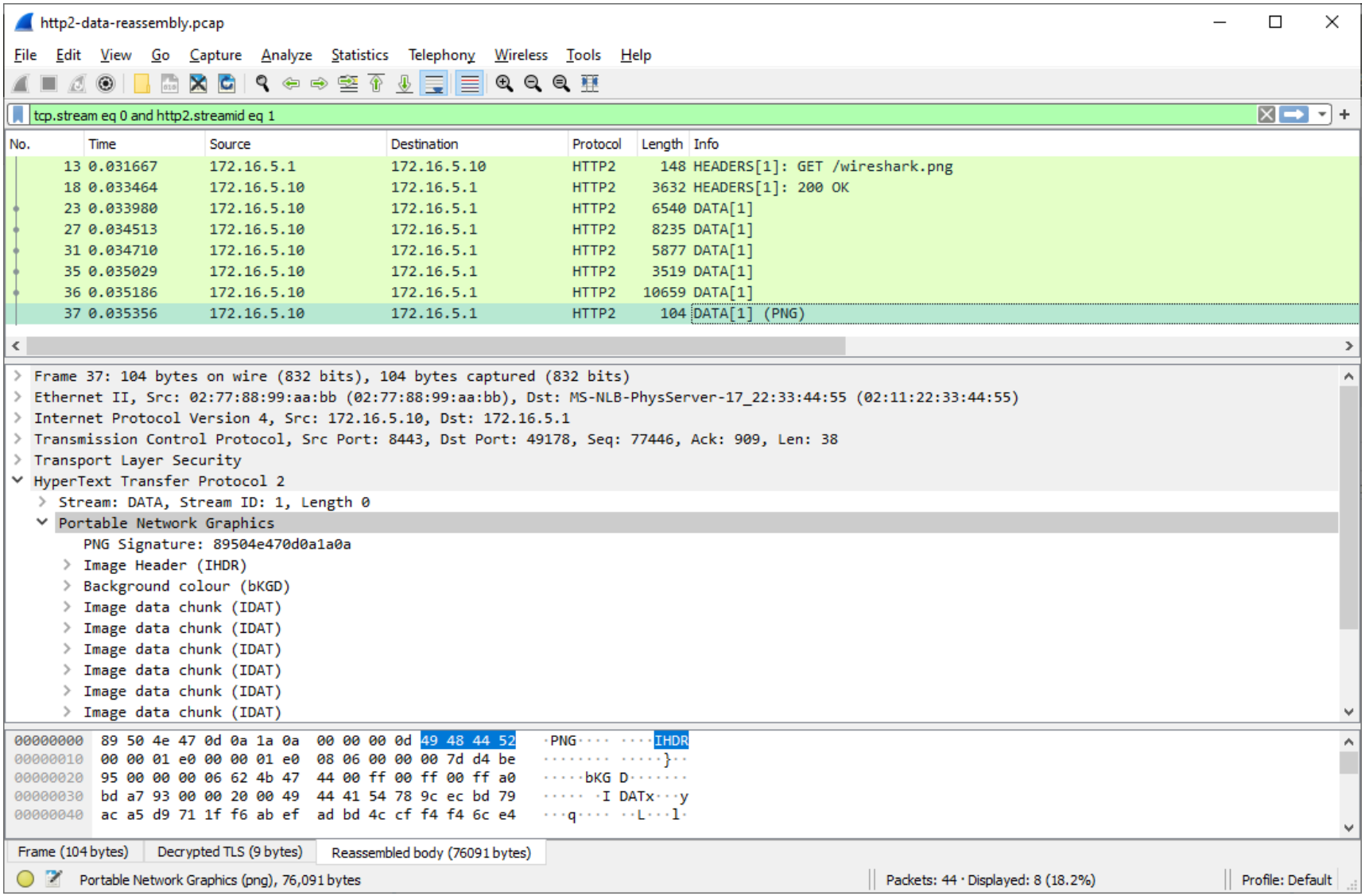
HTTP/2: Mitigating HOL blocking

- HTTP/2: objects divided into frames, frame transmission interleaved

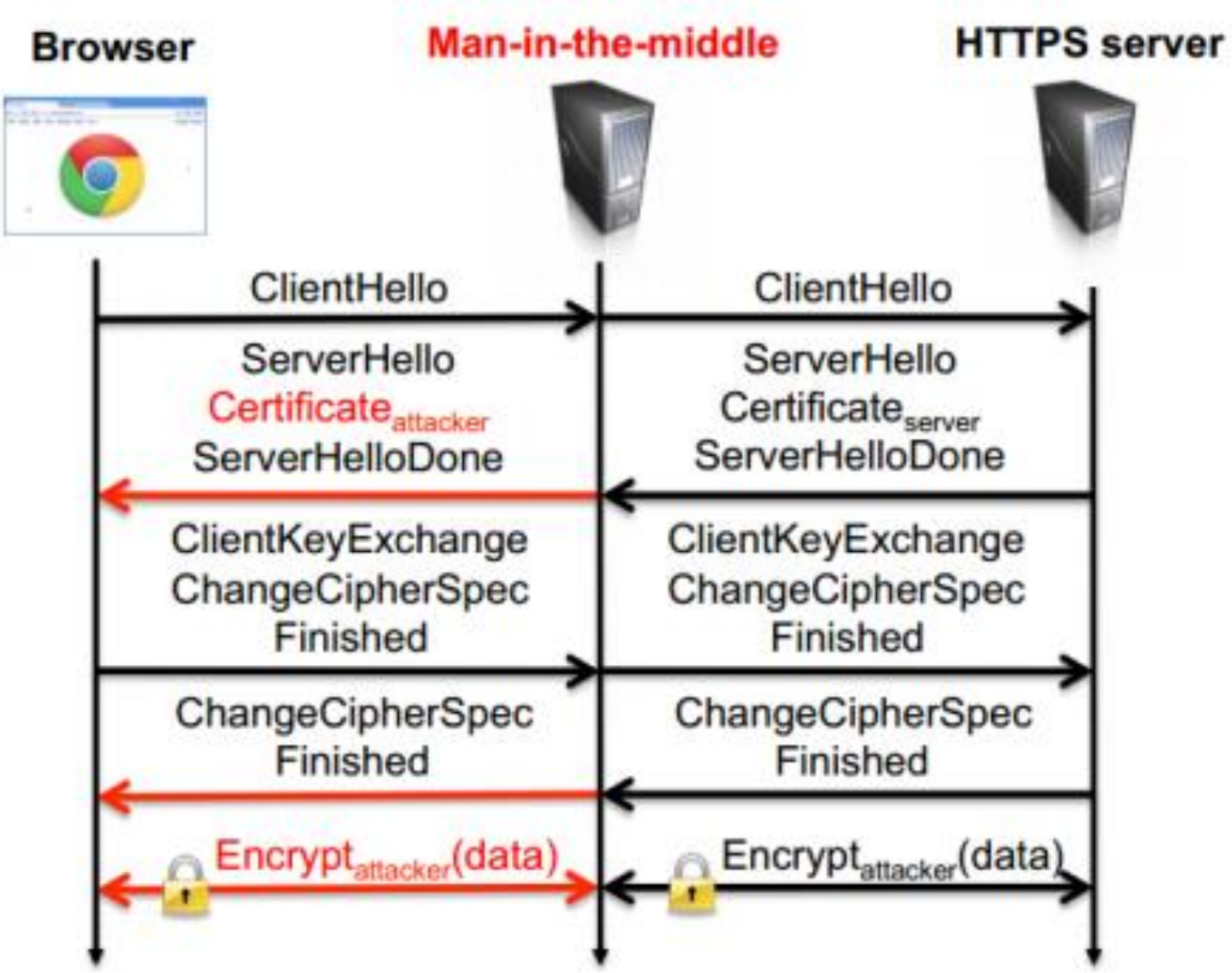


O_2, O_3, O_4 delivered quickly, O_1 slightly delayed

HTTP/2.0: Example



HTTPS DECODING DEMO


Browser
Man-in-the-middle
HTTPS server
ClientHello
ClientHello
ServerHello
Certificate attacker
ServerHelloDone
ServerHello
Certificate server
ServerHelloDone
ClientKeyExchange
ChangeCipherSpec
Finished
ClientKeyExchange
ChangeCipherSpec
Finished
ChangeCipherSpec
Finished
ChangeCipherSpec
Finished
Encrypt attacker(data)
Encrypt attacker(data)

Homework #1: Try to decode HTTP

- SSLSplit
  - https://github.com/nesfit/sslsplit_keylogger
  - https://www.signal-chief.com/2022/02/implementing-sslsplit/
- MITM Proxy
  - https://www.petergirnus.com/blog/decrypting-https-traffic-with-mitmproxy-amp-wireshark

Homework #2

1) Install Wireshark
2) Read about and install according to the preference
   - Hyper-V / VirtualBox / VMWare Workstation
   - Docker / LXC / Podman
   - GNS3, Eve-Ng, PNetLab
3) Reproduce demo with SSLsplit

DOMAIN NAME SYSTEM (DNS)

Domain Name System

- DNS is global directory of host names and other services
- It provides a system for translation of domain names into IP addresses

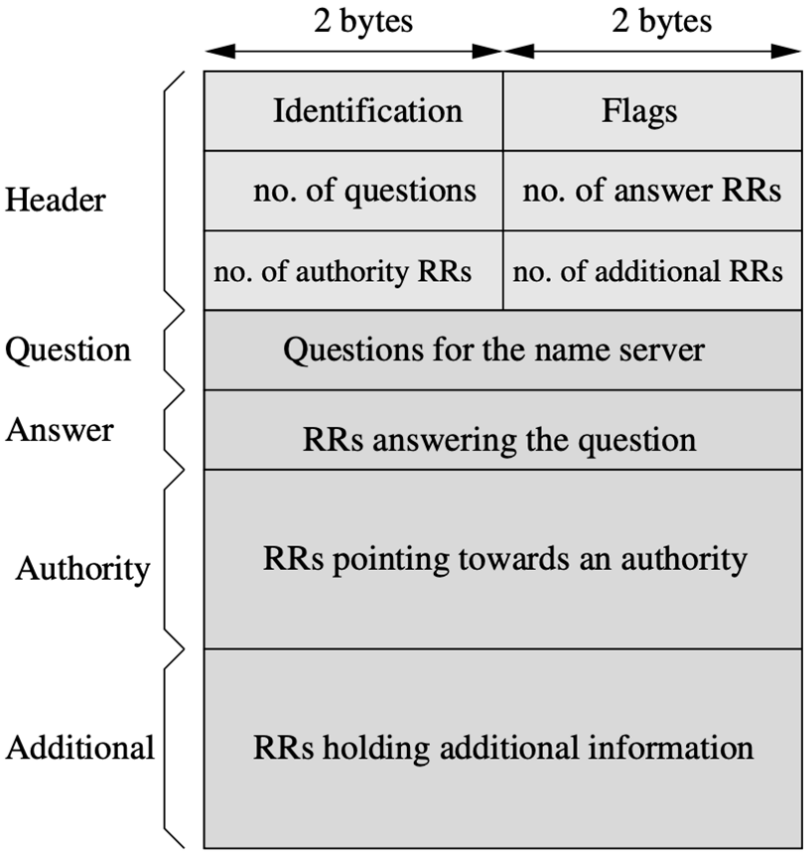
- **DNS architecture**
  - Namespace of DNS addresses + mapping of domain names to IP addresses
  - Storage of DNS namespace
  - Data access and lookups

- The domain names are organized in an inverted searching tree

- Domain names directory is physically stored on DNS servers
  - DNS servers contains information about tree structure
  - Each server manages only portion of domain names - a zone

- Type of servers:
  - **Primary (master, primary name servers)**
    - Contains full authoritative records about managed domains
    - Each domain only one primary server
  - **Secondary (slave, secondary name servers)**
    - Stores authoritative copies from primary servers
  - **Backup (caching-only name servers)**
    - Accept requests and forward to other servers
    - Cache responses
    - Offers non-authoritative responses (can be not actual)

- Introduced in RFC 1035
- Both underlying transport protocols UDP and TCP
  - Primarily UDP used
  - Port 53



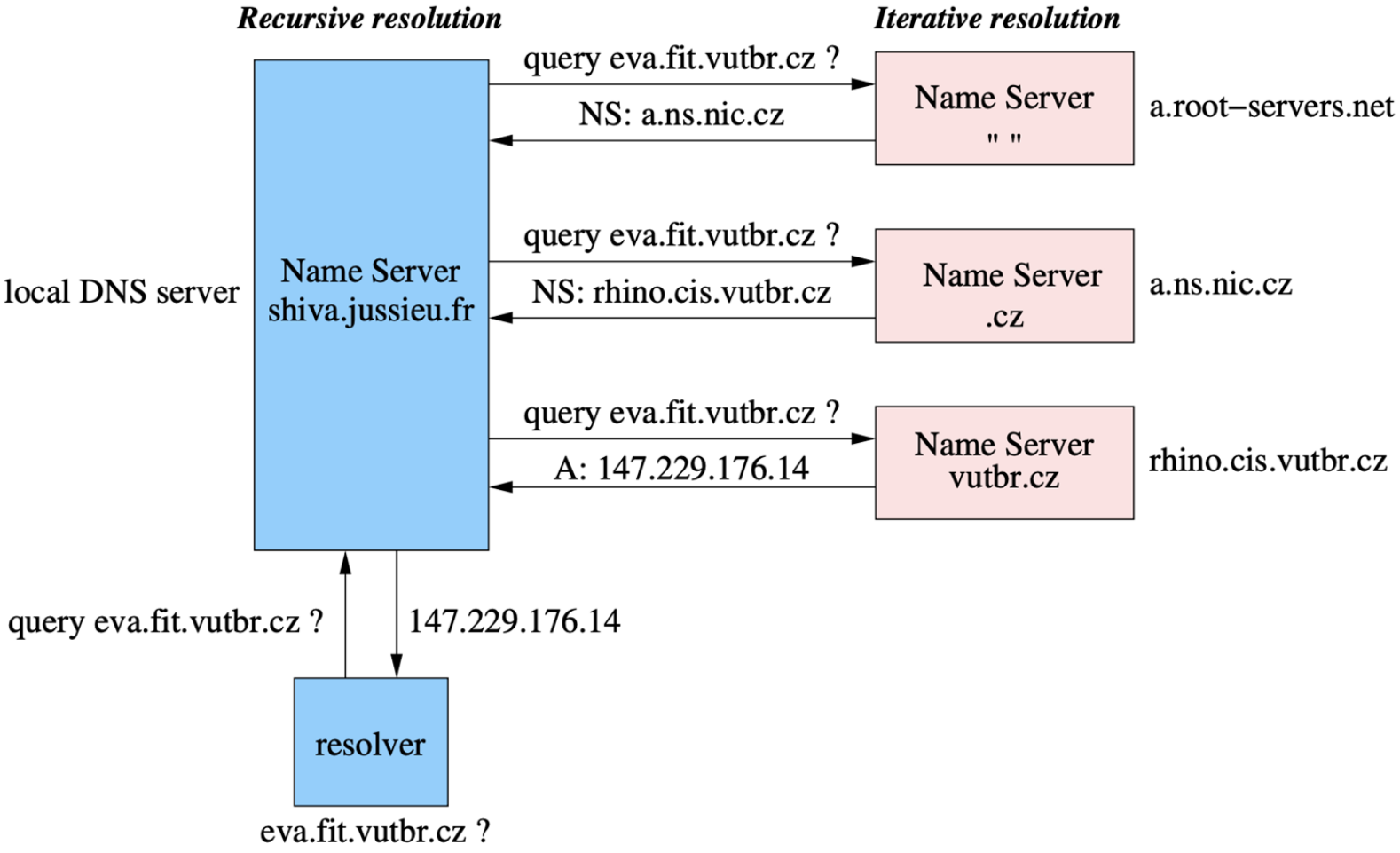
Example: answer to www.fit.vutbr.cz query

59058	qr, rd, ra, ad
1	1
4	6
www.fit.vutbr.cz IN A	
www.fit.vutbr.cz 13963 IN A 147.229.9.23	
fit.vutbr.cz. 13869 IN NS guta.fit.vutbr.cz. fit.vutbr.cz. 13869 IN NS kazi.fit.vutbr.cz fit.vutbr.cz 13869 IN NS rhino.cis.vutbr.cz fit.vutbr.cz 13869 IN NS gate.feec.vutbr.cz	
guta.fit.vutrbr.cz. 14062 IN A 147.229.8.11 kazi.fit.vutbr.cz. 13869 IN A 147.229.8.12 rhino.cis.futbr.cz. 86211 IN A 147.229.3.10 guta.fit.vutbr.cz. 14156 IN AAAA 2001:67c:1220:809 rhino.cis.vutbr.cz. 43692 IN AAAA 2001:67c:1220:e000:	

DNS Resolution

- Searching records in DNS
- **DNS Resolver**
  - Communication is of type client - server
  - System routine (resolver) resolves domain names for an application
- **DNS Resolution**
  - Process of searching answer for DNS query in DNS system
  - **Recursive Query**
    - If server do not know the answer, it sends the query to other servers
  - **Iterative Query**
    - If server do not know the answer, responds with a link to a server that can have the answer

Types of Resolution



Resource Records

- DNS Resource Records format
- Defined in RFC 1034, RFC 1035

Resource Records Format	Example
Name (variable length)	www.fit.vutbr.cz
Type (16 bits)	CNAME
Class (16 bits)	IN (0x0001)
TTL (32 bits)	4106 (1 h 8 min 26 s)
RDLENGTH (16 bits)	4
RDATA (variable length)	tereza.fit.vutbr.cz

Types of Records

- **A (Address)**
  - Direct mapping of domain name to IPv4 address
- **AAAA**
  - Direct mapping of domain name to IPv6 address
- **SOA (Start Of Authority)**
  - Every zone has only one SOA
  - Contains name of primary server, administrators address, several timers (Refresh, Retry, Expire, Minimum) and Serial identifying change of record
- **NS (Name Server)**
  - Determines authoritative server(s) for a zone
- **CNAME (Canonical Name)**
  - Maps alias to canonical name of a host
  - Alias is e.g. www, email, ...

Types of Records

- **PTR (Domain Name Pointer)**
  - Maps IPv4 and IPv6 address to domain name (reverse mapping)
- **NAPTR (Naming Authority Pointer)**
  - Dynamic configuration
  - Uses regular expressions for dynamic records
- **SRV (Service Record)**
  - Localization of services and servers, XMPP, SIP, ...
- **MX (Mail Exchanger)**
  - Contains mail servers to use for a domain
  - Multiple servers possible
- **TXT**
  - Extends the information about domain, server, administrator, ...

Tools For DNS Resolving

`whois <domain_name|IP>`

- Get the information about the owner of domain or IP address

`host|dig <domain_name>`

- Do the resolving for given domain name

`nslookup [OPTIONS] <domain_name>`

- OPTIONS
  - **`-type=<record_type_shorcut>`**
- example: **`nslookup -type=AAAA cisco.com`**

Nslookup Illustration

- `nslookup uba.ar 8.8.8.8`

```
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:    uba.ar
Address:  157.92.5.15
```

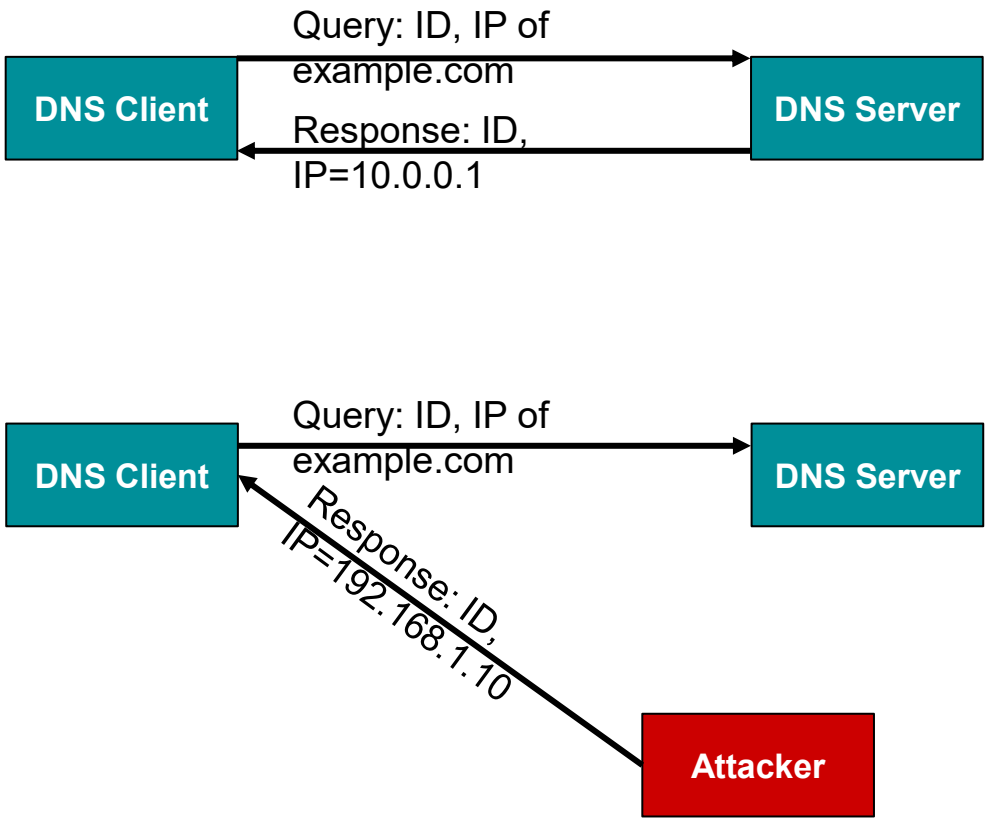
- `nslookup -type=MX fit.vut.cz 1.1.1.1`

```
Server:  one.one.one.one
Address:  1.1.1.1

Non-authoritative answer:
fit.vut.cz      MX preference = 10, mail exchanger = adela.fit.vut.cz
fit.vut.cz      MX preference = 20, mail exchanger = fiona.fit.vut.cz
```

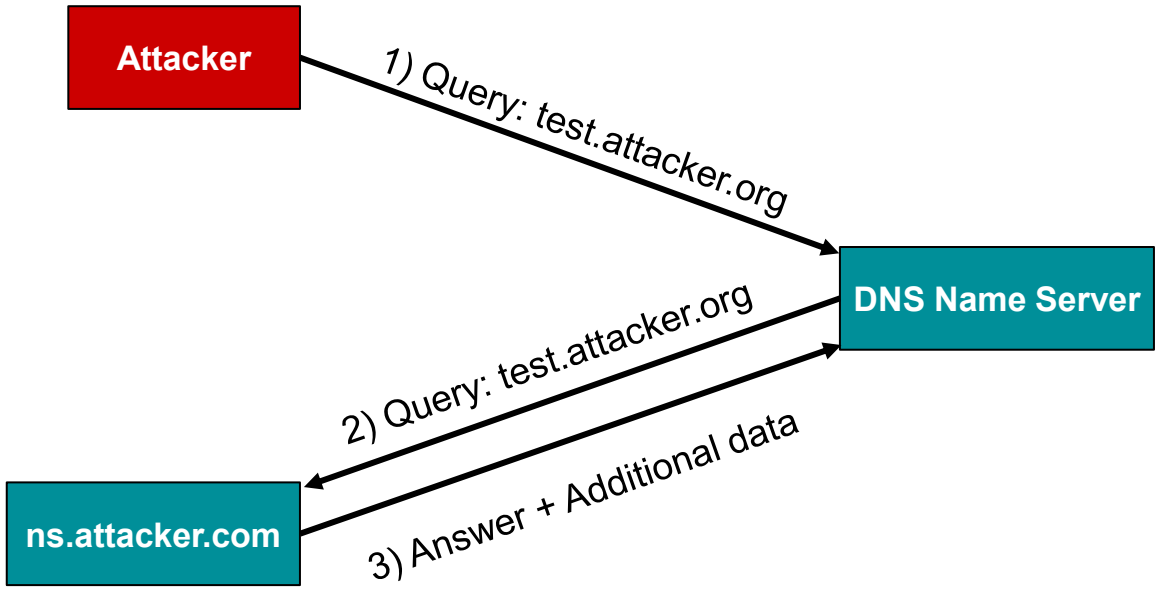
- All resolutions are stored in (local) caches

```
ipconfig /displaydns

Windows IP Configuration

    uba.ar
    ----------------------------------------
    Record Name . . . . . : uba.ar
    Record Type . . . . . : 1
    Time To Live  . . . . : 86396
    Data Length . . . . . : 4
    Section . . . . . . . : Answer
    A (Host) Record . . . : 157.92.5.15
```

- Unauthorized response for client query from the attacker

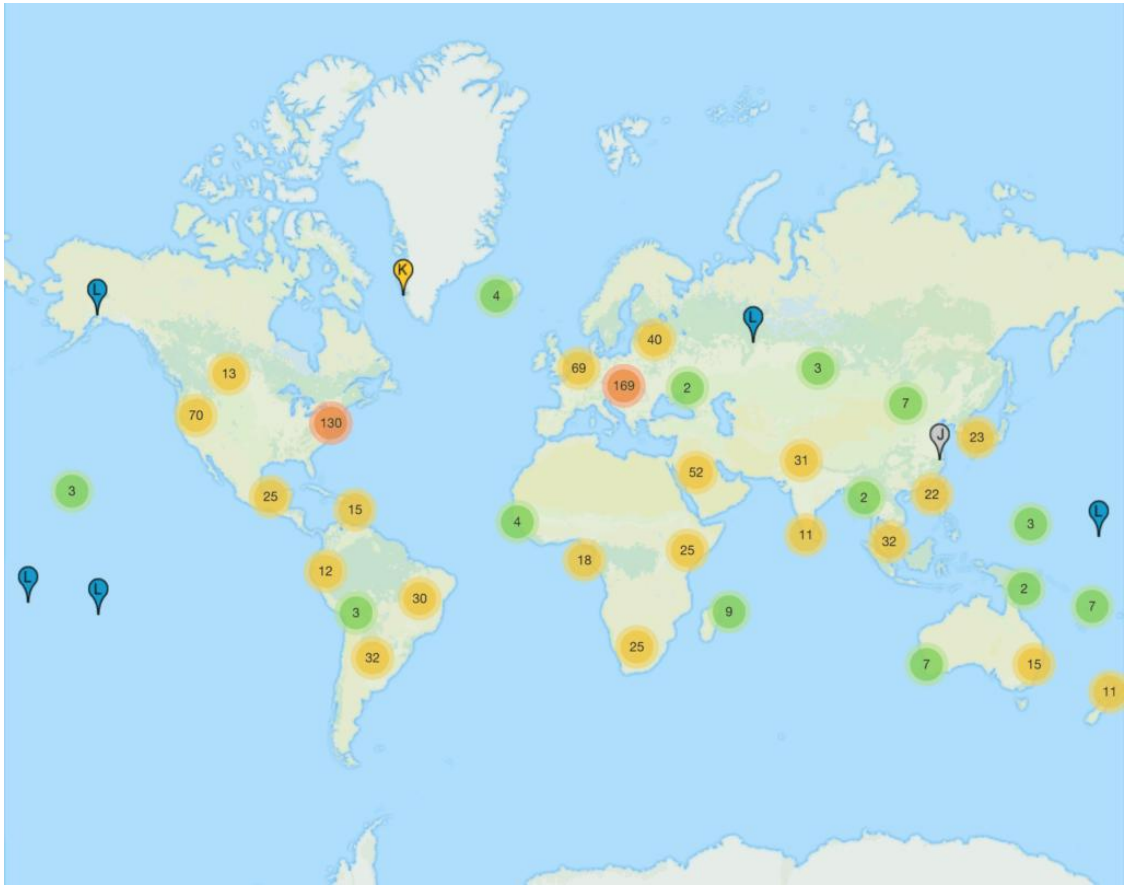


- DNS cache poisoning
- Insertion of incorrect information into cache



Vulnerabilities 3

- Distributed Denial of Service (DDoS)
- DDoS against root servers
- Blocking service for legitimate responses



Vulnerabilities 4

- DNS does not encrypt the traffic
- So anyone who can sniff the traffic can check the content of DNS packets
- This effect is used by network administrators to manage users
  - Point to right servers (in companies)
  - Or to filter the traffic
- But ISPs can also monitor (spy) the traffic
- Privacy issue

Fixing vulnerabilities 1, 2 and 3

SIGNING DNS RECORDS

- Covered by RFC 4033, RFC 4034, RFC 4035, RFC 5155
- Adds integrity and authentication to DNS data
- Asymmetric cryptography used for DNS record signing

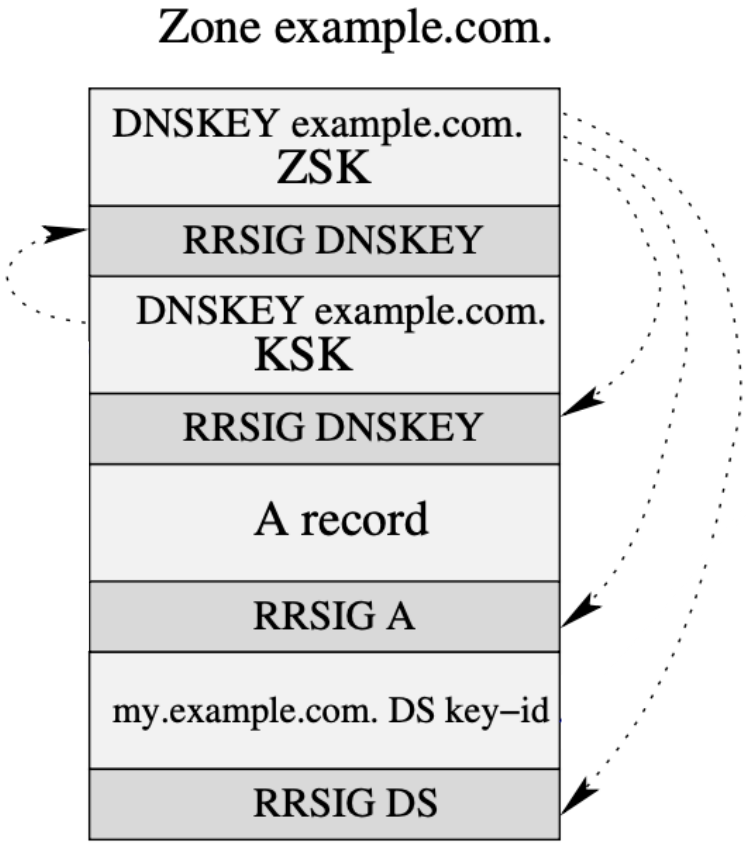
  - **Zone Signing Key (ZSK)**
    - Public, private key
    - Used for signing DNS records in zones

  - **Key Signing Key (KSK)**
    - Public, private key
    - Used for signing keys that are used for signing records

DNSSEC Records

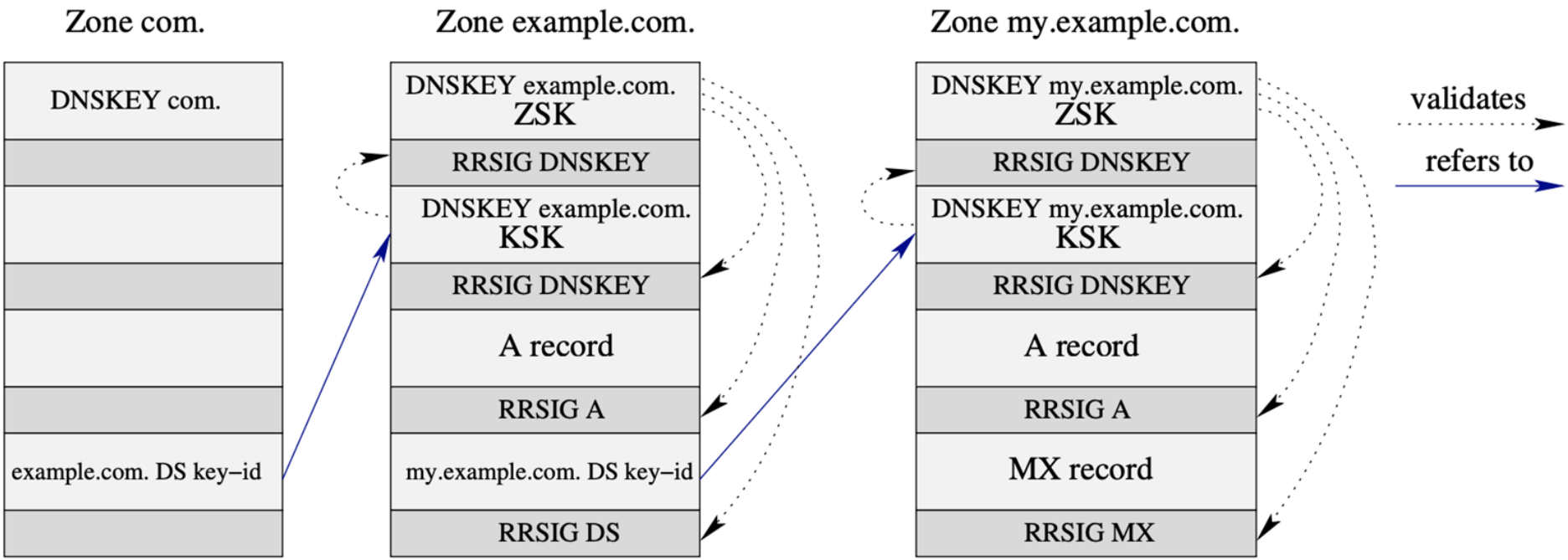
- New records are defined for DNSSEC
- **DNSKEY**
  - Public key for signature verification
- **RRSIG**
  - Signature of the record
- **NSEC, NSEC3**
  - Link to another record when the queried domain does not exists
- **DS**
  - Record for verification of DNSKEY record
  - Stored in superior domain

- **ZSK**
  - **Public Key**
    - Stored in DNSKEY record used to validate signatures in RRSIGs
  - **Private Key**
    - Used to sign zone records signatures stored in RRSIGs
- **KSK**
  - **Public Key**
    - Stored in DS record used to validate DNSKEY
  - **Private Key**
    - Used to sign DNSKEY



Signing Hierarchy

- Chain of trust
- Verification of key authenticity by the superior domain
- Public and Private key pair exists for every zone
- DNSSEC records are significantly increasing packet payload size

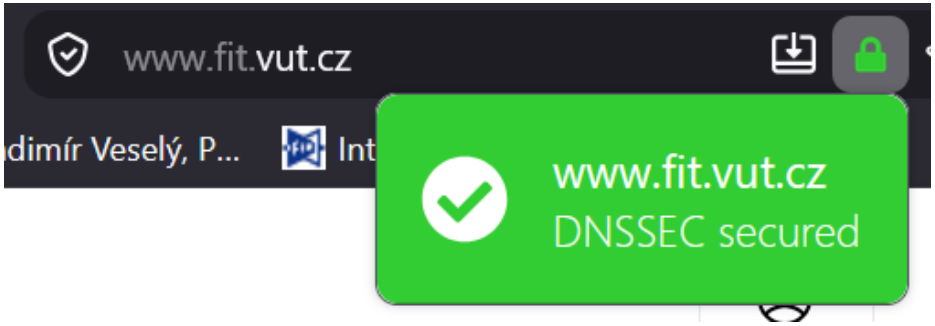


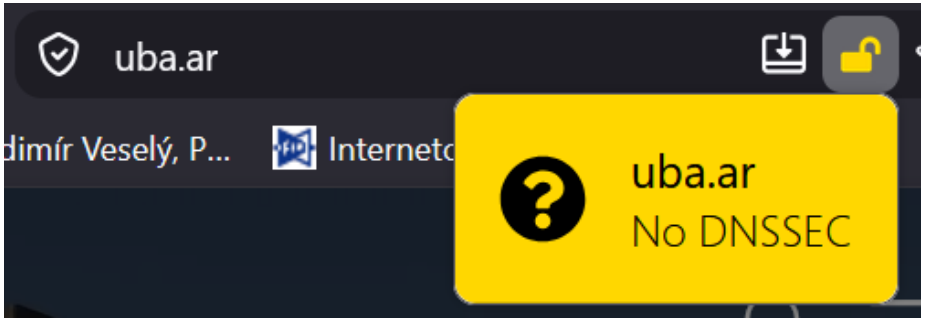
- **DNSSEC does not provide encryption!**
  - Everyone who will sniff the traffic can read the content of DNS messages
  - DNSSEC provides only integrity and authentication but not encryption
    - *It does not solves privacy issues!*

- *What can we use to encrypt DNS traffic?*
  - There are few attempts to encapsulate DNS messages to other encrypted protocols
  - DNS over TLS
  - DNS over HTTPS
  - DNS over QUIC

Check DNSSEC Status of Important Webpages

- Various add-ons and plug-ins
  - Unfortunately, nothing on OS level ☹



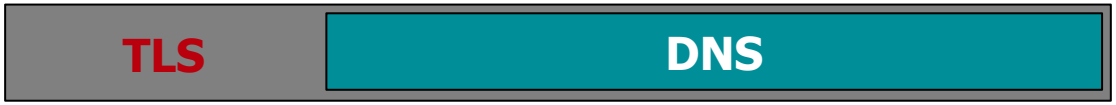


Fixing vulnerability 4

SECURING DNS TRAFFIC

DNS over TLS

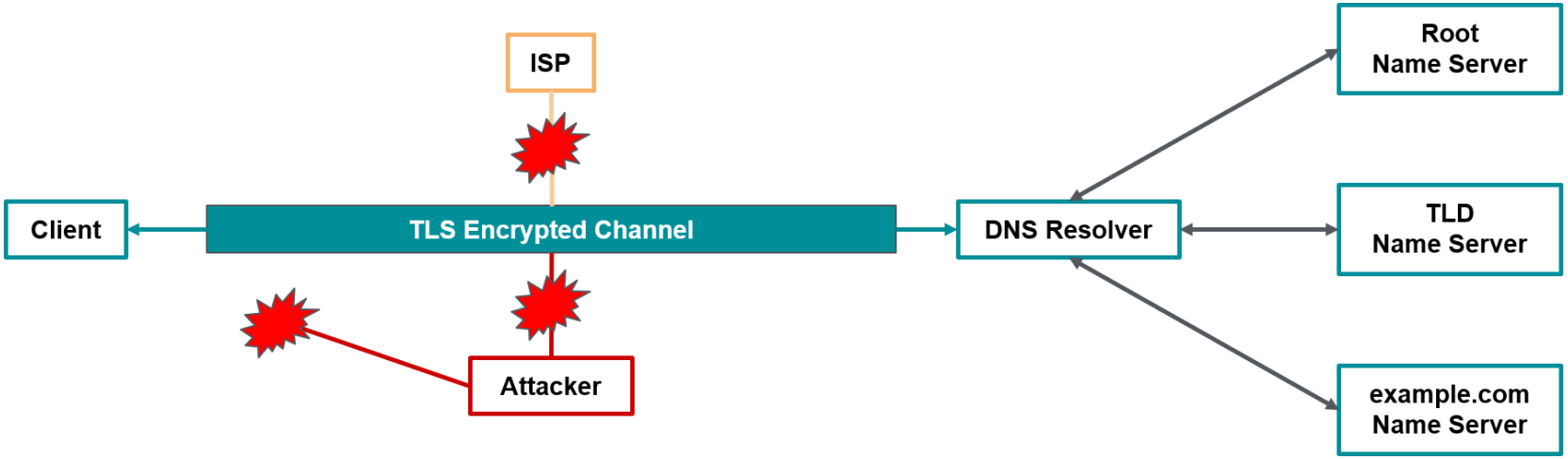
- Standard DNS queries and responses as defined in DNS are encapsulated by TLS protocol
- TLS protocol is used for encryption and transport of DNS messages
- Multiple queries can be transmitted over the same encrypted channel



- Defined in RFC 7858
- **Dedicated port 853/tcp**
- Support on the OS level, proxies and resolvers

DNS over TLS

- DoT moves the missing privacy to other party
- DNS traffic is encrypted only between Client (browser) and the DNS Resolver



- *Can we trust the DNS Resolver provider? Is it really private?*
- *Are those connections between DNS Resolver and Name Servers secure?*

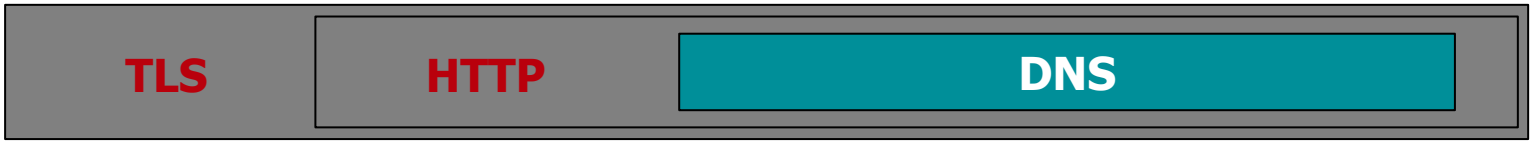
- `kdig`

```
$ kdig -d @1.1.1.1 +tls-ca +tls-host=cloudflare-dns.com  example.com
;; DEBUG: Querying for owner(example.com.), class(1), type(1), server(1.1.1.1), port(853), protocol(TCP)
;; DEBUG: TLS, imported 170 system certificates
;; DEBUG: TLS, received certificate hierarchy:
;; DEBUG:  #1, C=US,ST=CA,L=San Francisco,O=Cloudflare\, Inc.,CN=\*.cloudflare-dns.com
;; DEBUG:      SHA-256 PIN: yioEpqeR4WtDwE9YxNVnCEkTxIjx6EEIwFSQW+lJsbc=
;; DEBUG:  #2, C=US,O=DigiCert Inc,CN=DigiCert ECC Secure Server CA
;; DEBUG:      SHA-256 PIN: PZXN3lRAy+8tBKk2Ox6F7jIlnzr2Yzmwqc3JnyfXoCw=
;; DEBUG: TLS, skipping certificate PIN check
;; DEBUG: TLS, The certificate is trusted.
;; TLS session (TLS1.3)-(ECDHE-SECP256R1)-(ECDSA-SECP256R1-SHA256)-(AES-256-GC
;; ->>HEADER<<- opcode: QUERY; status: NOERROR; id: 58548
;; Flags: qr rd ra; QUERY: 1; ANSWER: 1; AUTHORITY: 0; ADDITIONAL: 1

;; EDNS PSEUDOSECTION:
;; Version: 0; flags: ; UDP size: 1536 B; ext-rcode: NOERROR
;; PADDING: 408 B

;; QUESTION SECTION:
;; example.com.             IN  A

;; ANSWER SECTION:
example.com.            2347    IN  A   93.184.216.34

;; Received 468 B
;; Time 2018-03-31 15:20:57 PDT
;; From 1.1.1.1@853(TCP) in 12.6 ms
```

- https://developers.cloudflare.com/1.1.1.1/dns-over-tls/

DNS over HTTPS

- Use TLS to encrypt the traffic
- Encapsulates DNS request/response by the HTTP
  - => HTTPS
- **Uses TCP transport port 443!**
  - The same port as for other HTTPS



DNS over HTTPS

- Defined in RFC 8484
- Brings encryption to DNS
- Can be enabled on the application level (browsers, proxies, tools)
- Currently supported in almost all browsers and mobile OS
- Hard to detect
  - Port based identification impossible
  - Employment of ML methods, blocklists, active scanning
- RFC recommended minimum HTTP/2
  - Adds additional establishment overhead

DoH POST Query

- HTTP accept type: "`application/dns-message`"
- Payload: DNS query in DNS Wireformat

```
:method = POST
:scheme = https
:authority = dnsserver.example.net
:path = /dns-query
accept = application/dns-message
content-type = application/dns-message
content-length = 33

<33 bytes represented by the following hex encoding>
00 00 01 00 00 01 00 00  00 00 00 00 03 77 77 77
07 65 78 61 6d 70 6c 65  03 63 6f 6d 00 00 01 00
01
```

- https://tools.ietf.org/html/rfc8484

DoH GET Query

- HTTP accept type: "`application/dns-message`"
- Query encoded using base64url without padding
  - Format: `/dns-query?dns=<base64url_encoded>`

```
:method = GET
:scheme = https
:authority = dnsserver.example.net
:path = /dns-query? (no space or Carriage Return (CR))
        dns=AAABAAABAAAAAAAAAWE-NjJjaGFyYWN0ZXJsYWJl (no space or CR)
        bC1tYWtlcy1iYXNlNjR1cmwtZGlzdGluY3QtZnJvbS1z (no space or CR)
        dGFuZGFyZC1iYXNlNjQHZXhhbXBsZQNjb20AAAEAAQ
accept = application/dns-message
```

DoH Query Response

- Standardised (RFC 8484)
- HTTP accept type: "`application/dns-message`"
- Success 2xx with query response
- Unsuccessful response 4xx without any query response

```
:status = 200
content-type = application/dns-message
content-length = 61
cache-control = max-age=3709

<61 bytes represented by the following hex encoding>
00 00 81 80 00 01 00 01  00 00 00 00 03 77 77 77
07 65 78 61 6d 70 6c 65  03 63 6f 6d 00 00 1c 00
01 c0 0c 00 1c 00 01 00  00 0e 7d 00 10 20 01 0d
b8 ab cd 00 12 00 01 00  02 00 03 00 04
```

DoH GET JSON

- **Not standardised**
- Cloudflare,Google - similar request, the same response
- Http accept type “application/dns-json”

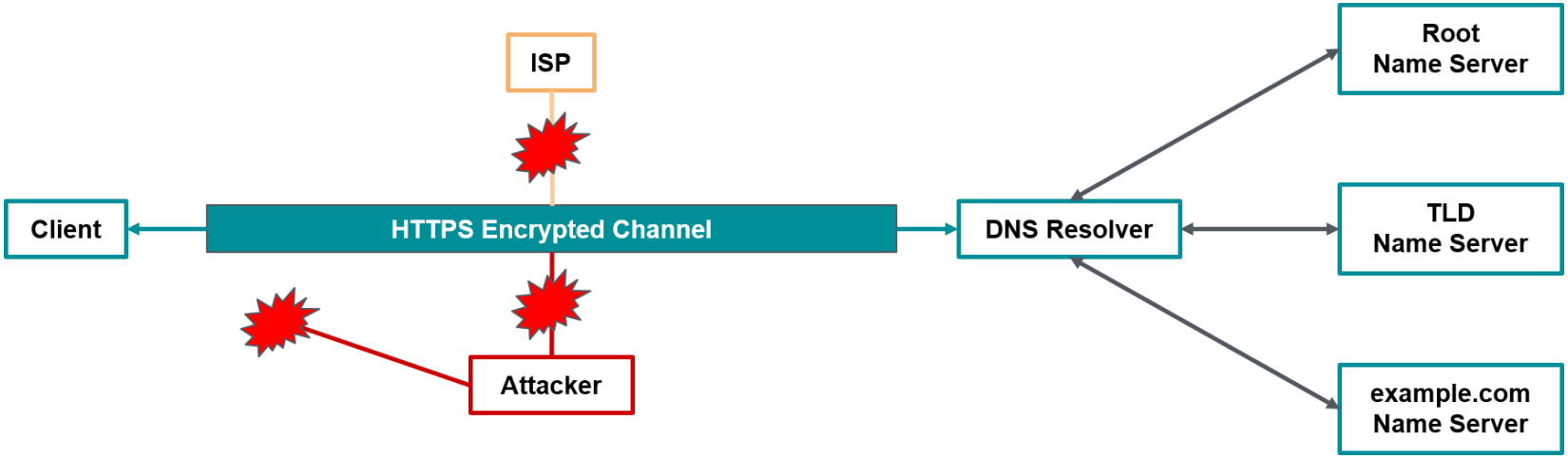
```
curl -H 'accept: application/dns-json' 'https://cloudflare-dns.com/dns-query?name=example.com&type=AAAA'
```


```
{
  "Status": 0,
  "TC": false,
  "RD": true,
  "RA": true,
  "AD": true,
  "CD": false,
  "Question": [
    {
      "name": "example.com.",
      "type": 28
    }
  ],
  "Answer": [
    {
      "name": "example.com.",
      "type": 28,
      "TTL": 1726,
      "data": "2606:2800:220:1:248:1893:25c8:1946"
    }
  ]
}
```


- https://developers.cloudflare.com/1.1.1.1/dns-over-https/json-format/

DNS over HTTPS

- DoH provides similar functionality as DoT
- DoH has higher payload overhead, longer initiation phase
- Does it solves any of the problems of DoT?

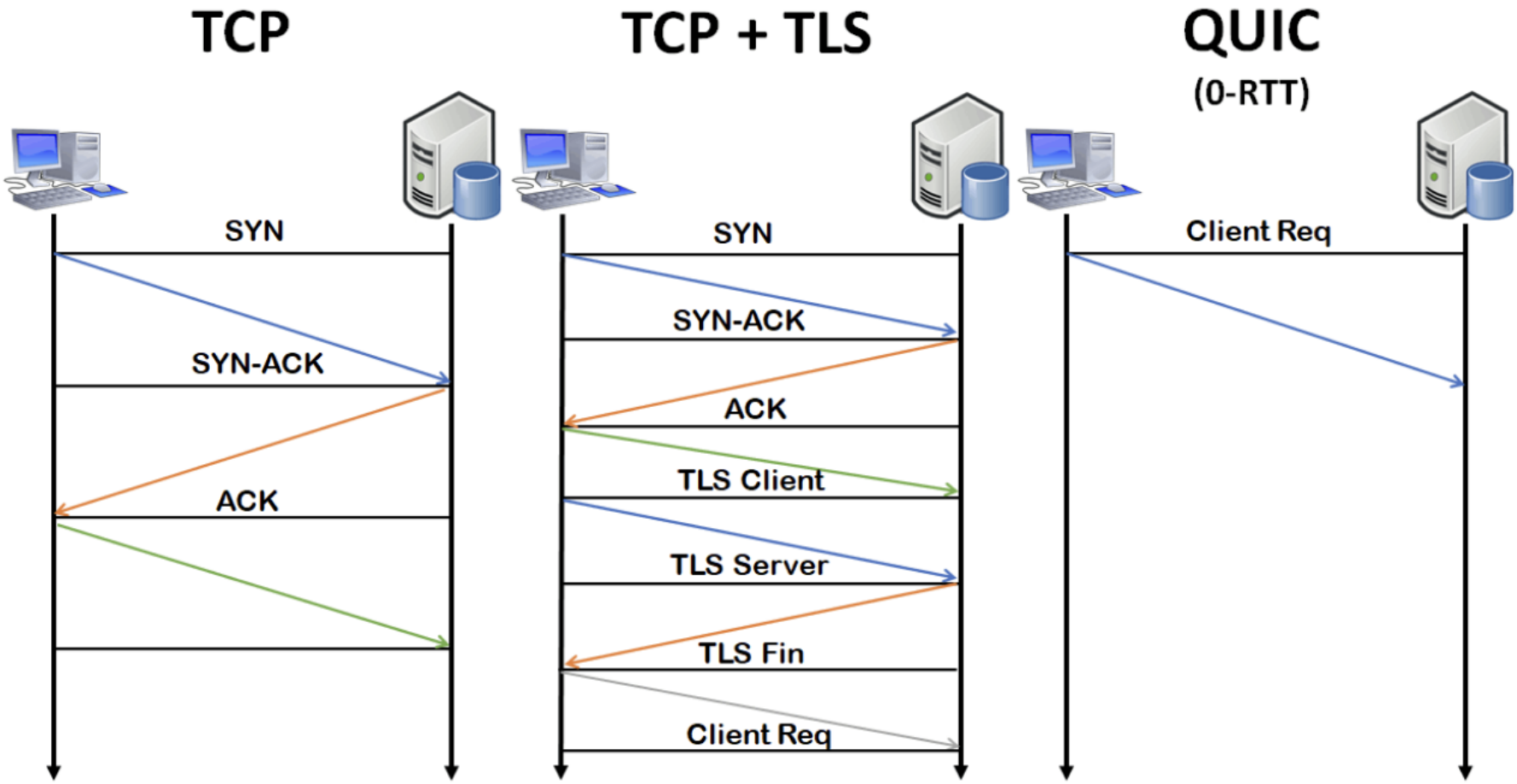


- Similar privacy concerns remains:
  - *Can we trust the DNS Resolver provider? Is it really private?*
  - *Are those connections between DNS Resolver and Name Servers secure?*

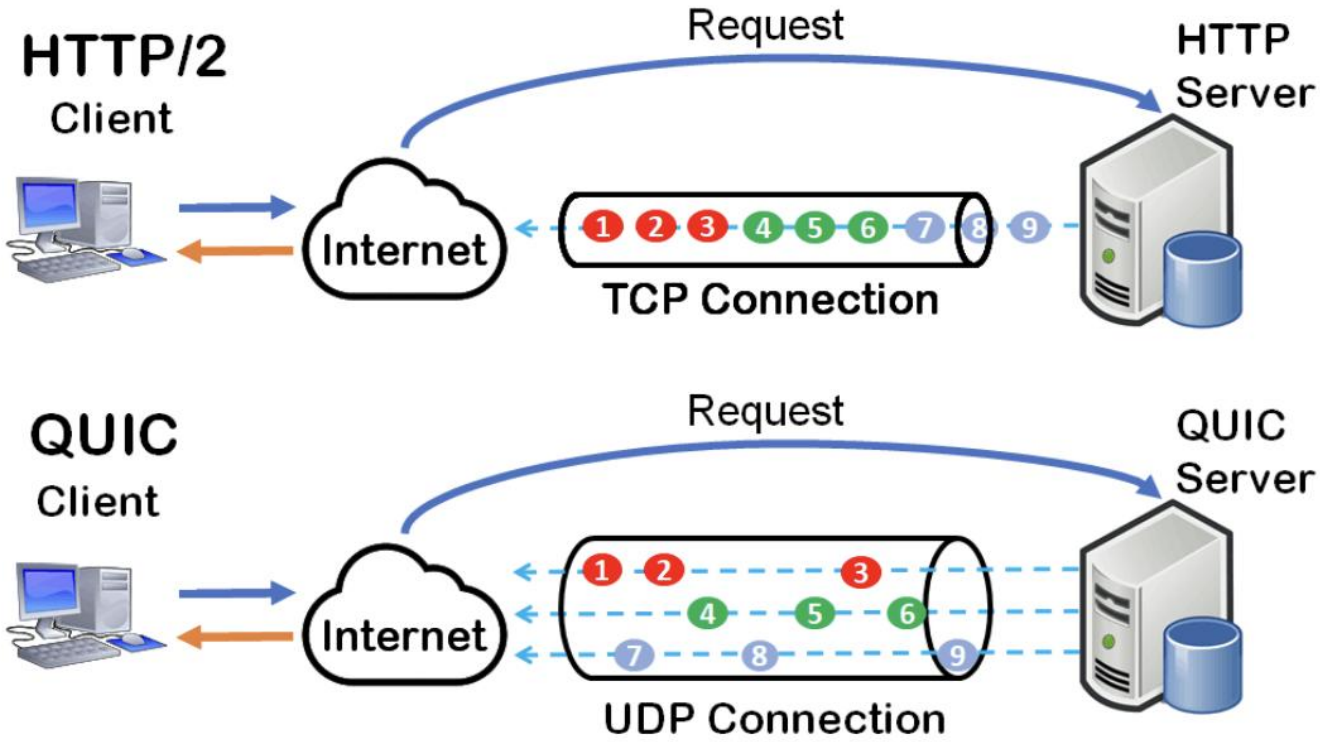
DNS over QUIC

- Defined in RFC 9250
- Use QUIC to transport DNS
- **Use dedicated port 853/udp**

Why QUIC?

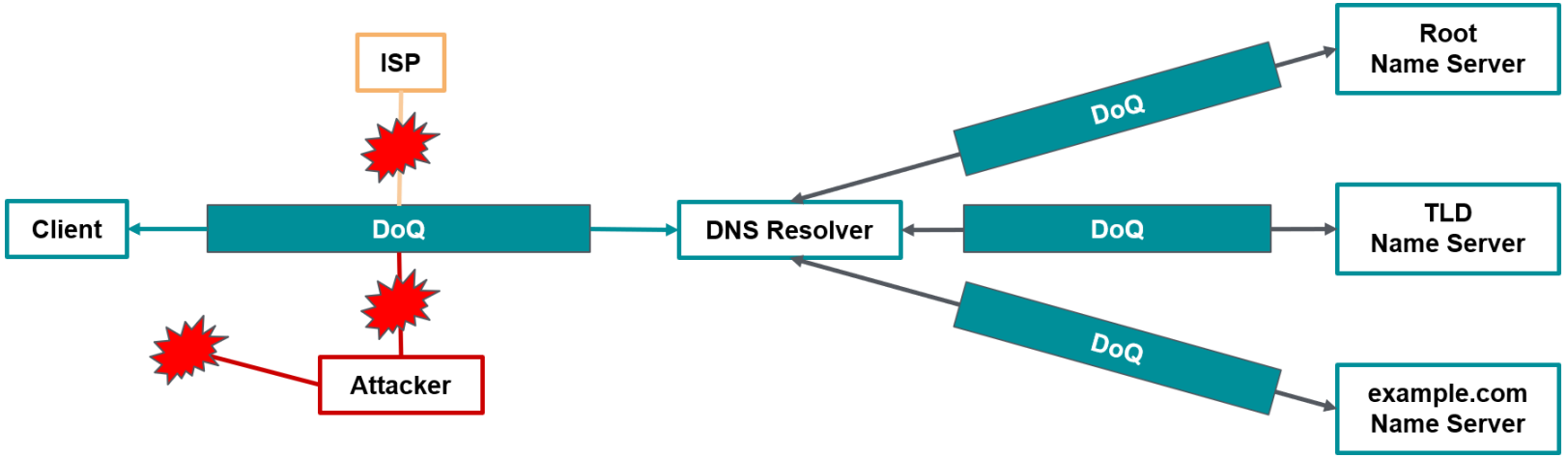

TCP
SYN
SYN-ACK
ACK
TCP + TLS
SYN
SYN-ACK
ACK
TLS Client
TLS Server
TLS Fin
Client Req
QUIC
(0-RTT)
Client Req

- Ahead-of-line blocking
- https://avenash-kumar.com/blog/quic-birth-of-new-protocol-in-layer4-family



DNS over QUIC

- DoQ provides similar functionality as DoT but even more
- DoQ has less payload overhead than DoH
- Does it solves any of the problems of previous mentioned?



- Similar privacy concerns remains:
  - *Can we trust the DNS Resolver provider? Is it really private?*
  - *Are those connections between DNS Resolver and Name Servers secure?*

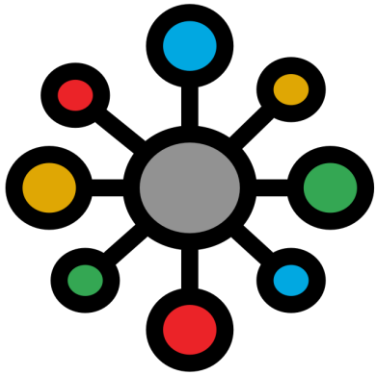
Q & A

Self-test

- What is PKI? Describe its components.
- What is certificate? What does it contain? Where is it used?
- What is TLS? On which layer it operates? What messages are being exchanges during TLS handshake?

- RISTIC, Ivan. *Bulletproof SSL and TLS: Understanding and deploying SSL/TLS and PKI to secure servers and web applications*. Feisty Duck, 2022.

Vladimír Veselý veselyv@fit.vut.cz



- https://www.fit.vut.cz/research/groups/nes@fit/
- https://git.fit.vutbr.cz/NESFIT
- https://nesfit.github.io